Capítol 2. Especificació de sistemes de programari

Autor: Xavier Franch. Revisors: Dolors Costal, Lidia López

UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Facultat d'Informàtica de Barcelona

Índex

2.1. Enginyeria de requisits

2.2. Model de l’especificació

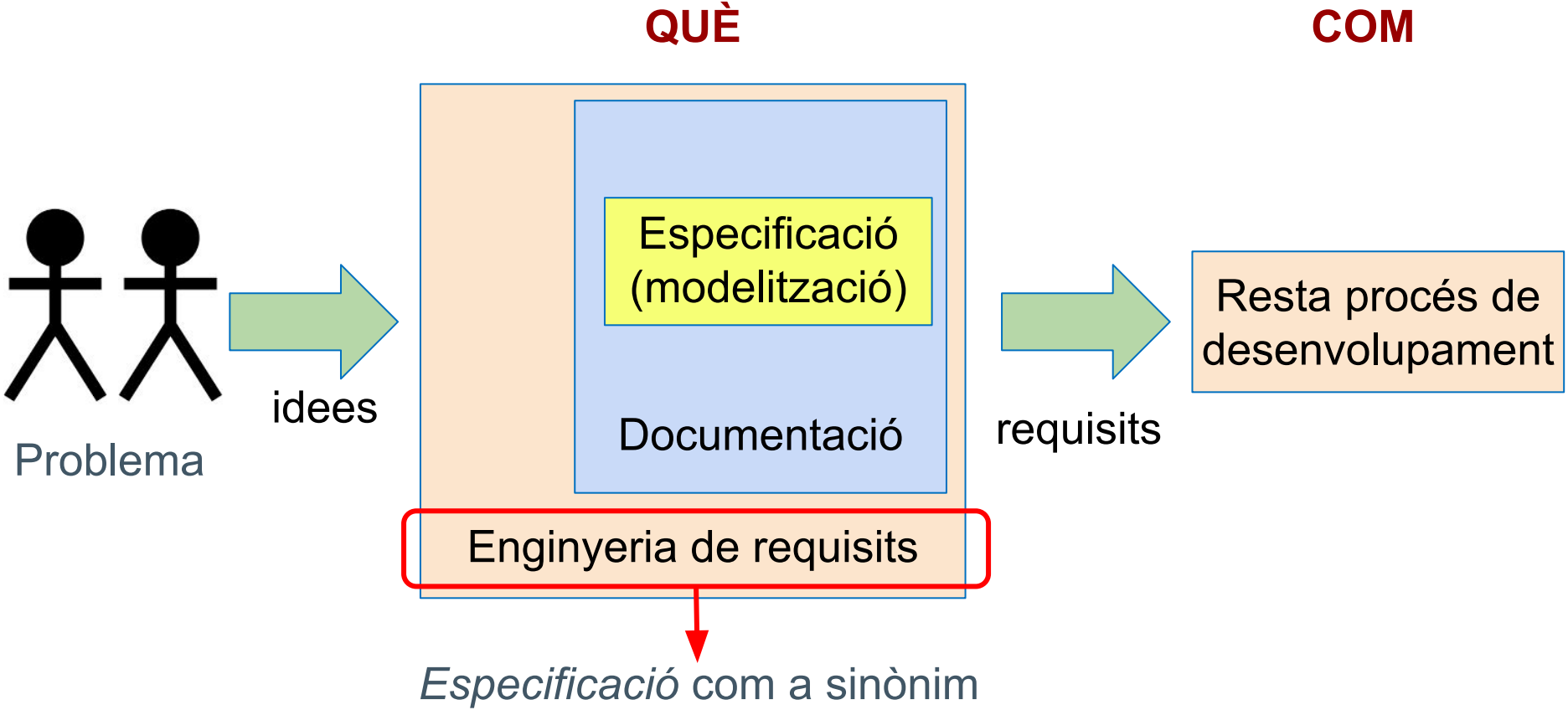
Preliminars

Al llarg d’aquest capítol, usarem un exemple per il.lustrar els conceptes introduïts: sistema de músics i concerts

Volem un sistema que guardi músics amb els instruments que toquen, grups musicals i concerts que donen aquests músics i/o grups

2.1 Enginyeria de requisits

Context


QUÈ
COM
Especificació (modelització)
Documentació
Enginyeria de requisits
Problema
idees
requisits
Resta procés de desenvolupament
Especificació com a sinònim

Enginyeria de requisits (ER)

Aproximació sistemàtica a l'especificació i gestió de **requisits** amb l'objectiu de comprendre les necessitats i desitjos de les **parts implicades** en el sistema

Spice Girls. Wannabe (1996)

Enginyeria de requisits (ER)

Aproximació sistemàtica a l'especificació i gestió de **requisits** amb l'objectiu de comprendre les necessitats i desitjos de les **parts implicades** en el sistema

Requisit:

1. **Necessitat** percebuda per una part implicada
2. **Capacitat** o propietat que el sistema ha de tenir
3. **Representació** documentada d'una necessitat, capacitat o propietat

Exemples de requisits

El sistema ha de permetre a un client comprar fins a un màxim de 10 entrades per concert

Per cada client, el sistema guarda el seu dni, nom i adreça electrònica

En completar la informació necessària per a la compra, el sistema ha de mostrar al client un formulari amb camps per al número, data d'expiració i CVV de la tarja de crèdit

El sistema ha d'assegurar que les dades de la tarja de crèdit no queden exposades en cap moment

En el processament total de la compra d'entrades, el sistema ha de donar un temps de resposta menor de 5 segons en el 99,5% dels casos

Enginyeria de requisits (ER)

Aproximació sistemàtica a l'especificació i gestió de **requisits** amb l'objectiu de comprendre les necessitats i desitjos de les **parts implicades** en el sistema

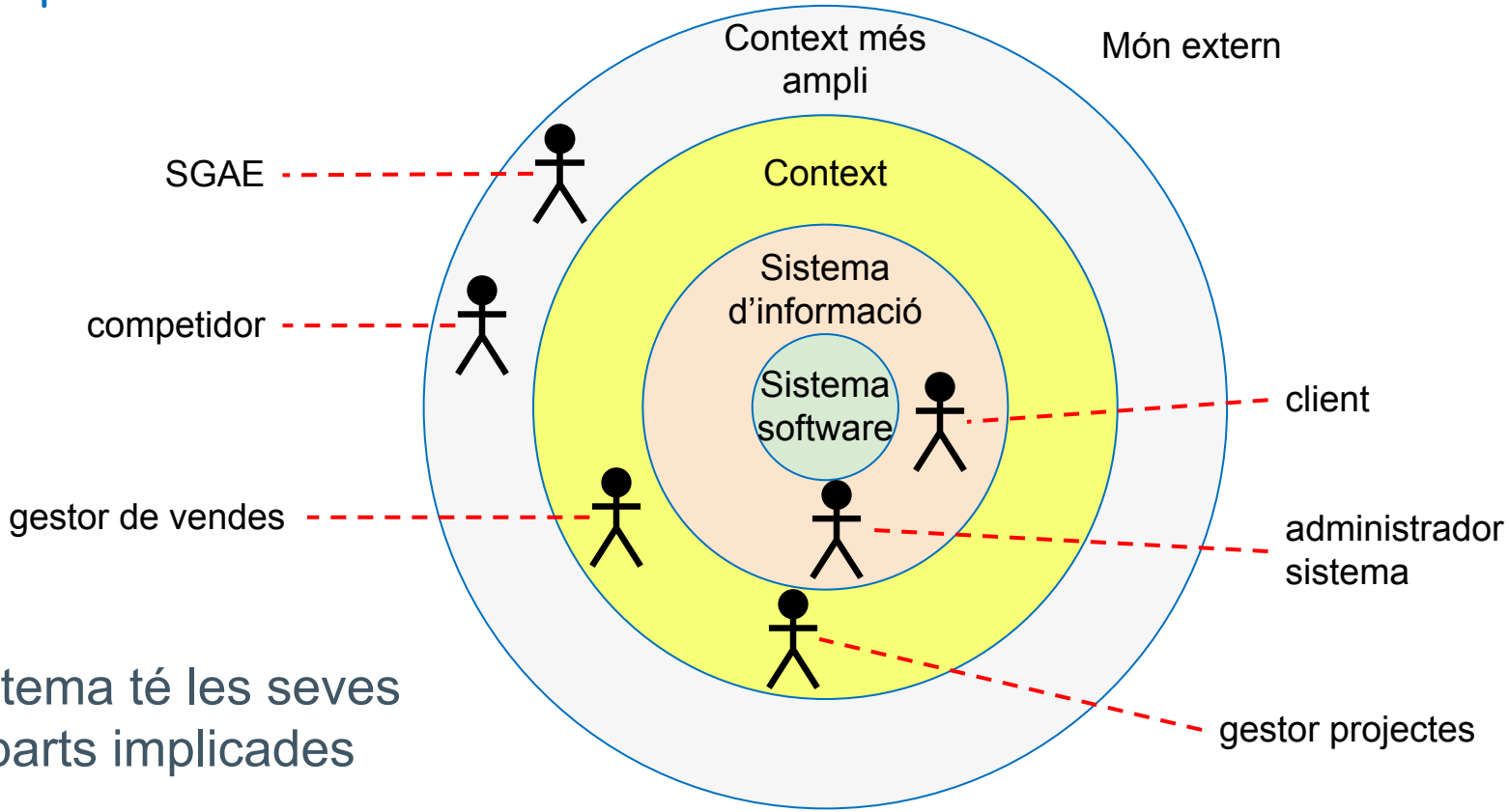
Requisit:

1. **Necessitat** percebuda per una part implicada
2. **Capacitat** o propietat que el sistema ha de tenir
3. **Representació** documentada d'una necessitat, capacitat o propietat

Part implicada (*stakeholder*): persona o organització que:

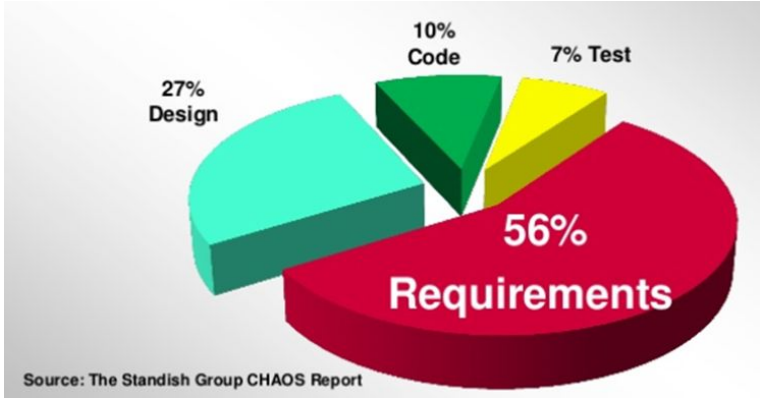
1. **Influeix** en els requisits del sistema, o bé
2. **Rep** l'impacte del sistema

Parts implicades (*stakeholders*)



Cada sistema té les seves pròpies parts implicades

Importància de l'ER



https://www.gartner.com/en/documents/2810920

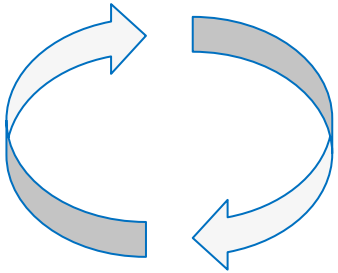
ER com a disciplina : Principis fonamentals

ER és mitjà per a un fi

Treball sistemàtic

Satisfer parts implicades

Requisits són canviants



Compartir visió

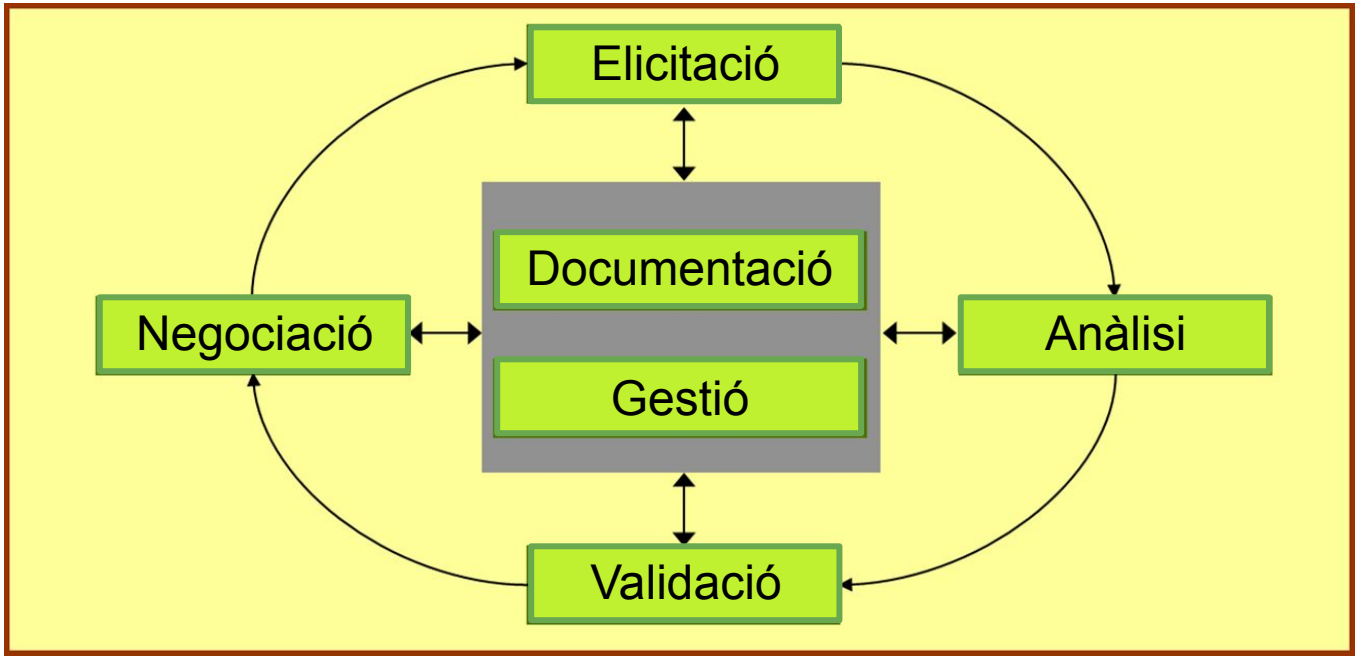
Cercar innovació

Considerar context

Validació necessària

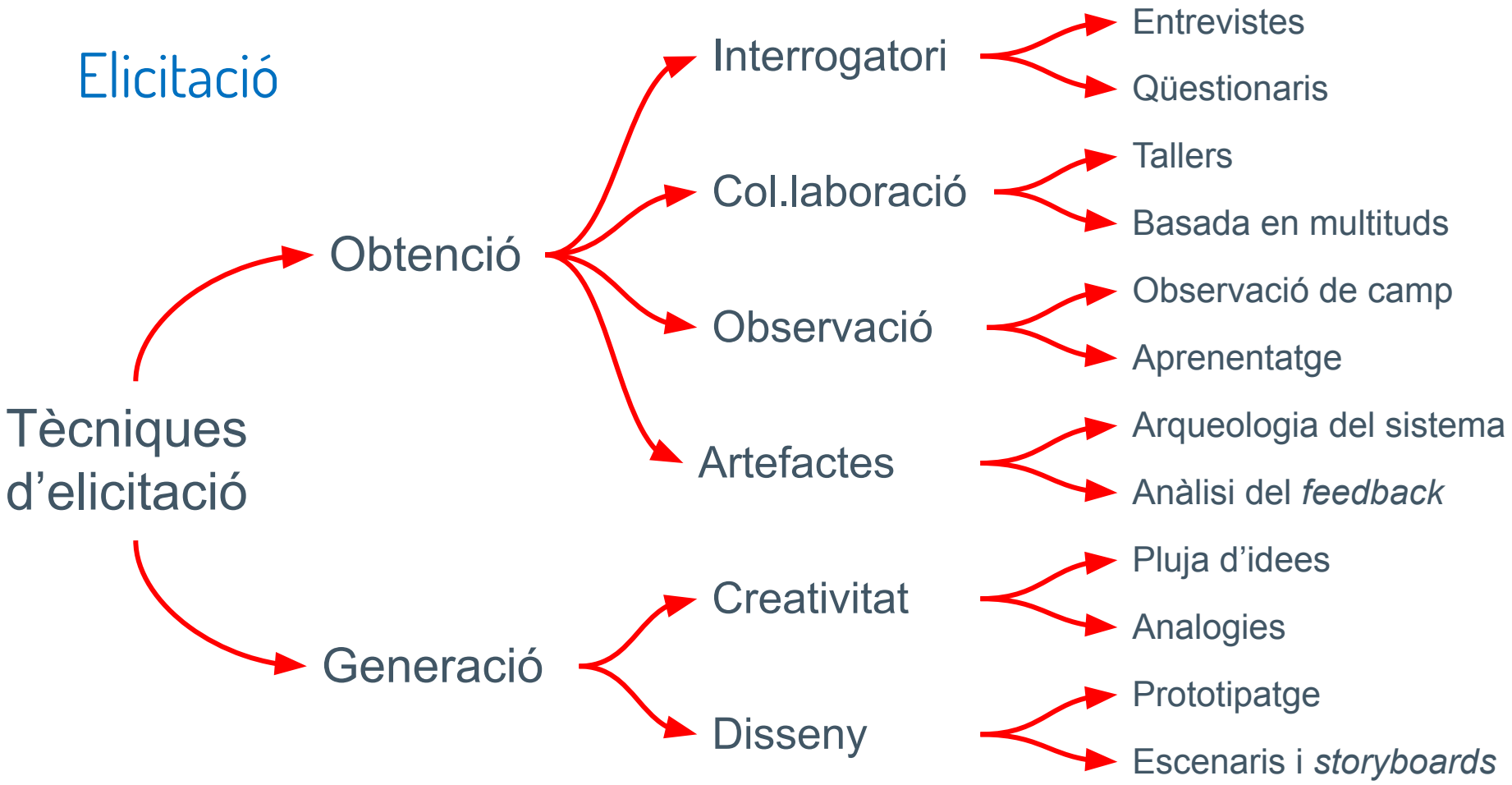
Problema-Req.-Solució

Activitats de l'ER



Ian Sommerville. Integrated Requirements Engineering: A Tutorial. IEEE Softw. 22(1): 16-23 (2005)

Elicitació


Tècniques d'elicitació
Obtenció
Generació
Interrogatori
Col.laboració
Observació
Artefactes
Creativitat
Disseny
Entrevistes
Qüestionaris
Tallers
Basada en multituds
Observació de camp
Aprenentatge
Arqueologia del sistema
Anàlisi del *feedback*
Pluja d'idees
Analogies
Prototipatge
Escenaris i *storyboards*

Anàlisi

Cal comprovar que els requisits compleixen certes propietats

Requisits individuals	Conjunt de requisits
Adequat	Consistent
Necessari	No redundant
No ambigu	Complet (sota cert criteri)
Complet	Traçable
Comprensible	Conforme (a estàndards, formats, ...)
Validable	

ISO/IEC/IEEE 29148:2018. Systems and software engineering — Life cycle processes — Requirements engineering

Validació

Cal validar:

- els requisits cobreixen les **necessitats** de les parts implicades
- grau d'**acord** entre les parts implicades
- probabilitat dels **supòsits** sobre el context

Tècniques:

- **revisions**:
  - *walkthrough*: revisió conjunta per l'equip de treball
  - inspecció: revisió delegada a un actor extern
- **exploració**:
  - construcció de prototipus
  - alpha/beta testing, A/B testing

Negociació

Diverses parts implicades tenen necessitats diferents:

- cal **prioritzar** els requisits derivats
- cal identificar i resoldre **conflictes**

Priorització:

- model **Kano**:
  - classifica necessitats en *delighters*, *satisfiers* i *dissatisfiers*
- qualsevol altra escala ordinal (números, categories, …)

Conflictes: diverses accions són possibles

- acord, compromís, votació, imposició; altres més específiques de l'ER
- també pot afectar a l'especificació del sistema (e.g., perfils, opcions)

Gestió

Cal gestionar (amb ajut d'eines especialitzades si és possible):

- els requisits **individuals**
- el **conjunt** de requisits (en forma de documents)
- les **relacions** dels requisits amb altres parts del sistema

Aspectes clau:

- **cicle de vida**
  - e.g., elicitat -> documentat -> validat -> implementat -> discontinuat
- **control de versions** i **configuracions**
- **metadades**: atributs associats als requisits (e.g., origen, prioritat)
- **traçabilitat**: relacions entre requisits, i amb altres productes

Documentació

Diversos estils són possibles

- **formal**: basat en algun tipus de lògica
  - permet fer demostracions
  - restringit a àmbits molt concrets (típicament, *life-critical systems*)
- **semi-formal**: basat en diagrames que representen models
  - permet anàlisi i validació sistemàtiques
  - bon compromís en un gran nombre de casos
- **informal**: basat en llenguatge natural
  - sovint seguint algun tipus de plantilla
  - fàcil d'usar però font d'errors futurs

És usual combinar aquests estils en un mateix sistema

Documentació en llenguatge natural

Composat de:

- **document de requisits**:
  - tradicionalment, també es diu especificació de requisits (tot i que no sigui formal) (SRS per inicials en anglès)
  - conté diverses parts recurrents, segons certa estructura
  - conté tots els requisits, classificats segons certa taxonomia
  - pot anar acompanyat d'esquemes o figures, o ocasionalment d'algun model en alguna de les seves parts
  - els requisits individuals, normalment segons alguna plantilla més o menys genèrica
- **glossari**: per compartir un vocabulari entre totes les parts implicades
  - simplement, un diccionari

Escriptura dels requisits

Regles recomanades:

- un requisit = una frase en llenguatge natural
- les frases han de ser curtes i directes
  - definir plantilles (n'existeixen unes quantes)
- sempre basar-se en el glosari (que es va construint incrementalment)
- evitar:
  - ús de veu passiva (doncs, qui fa què?)
  - quantificadors universals (*tots*, *sempre*, *mai*, …)
  - noms no específics (*l'usuari*, *les dades*, …)
  - comparatives incompletes (...*millor temps de resposta*…; respecte de què?)

Tipus de requisits

Distingim:

- **requisits funcionals**: descriuen funcions del sistema
- **requisits de qualitat**: descriuen aspectes relacionats amb la qualitat del sistema
- **restriccions**: requisits que limiten l'espai de la solució

de vegades s'anomenen simplement **requisits no funcionals**

Aquests tipus no són sempre fàcils de distingir l'un de l'altre

Requisits funcionals

Distingim:

- **requisits de dades**: descriuen les dades que gestiona el sistema
  - e.g.: *Per cada client, el sistema guarda el seu dni, nom i adreça*
- **requisits de funció**: descriuen les funcionalitats que el sistema ofereix, eventualment mostrant com les funcions es comuniquen
  - e.g.: *El sistema ha de permetre a un client comprar fins a un màxim de 10 entrades per concert*
- **requisits de comportament**: descriu la reacció del sistema a esdeveniments externs, possiblement en funció de l'estat actual
  - e.g.: *En completar la informació necessària per a la compra, el sistema ha de mostrar al client un formulari amb camps per al número, data d'expiració i CVV de la tarja de crèdit*

Requisits de qualitat - natura

- **qualitatiu**: ambigu, difícil de validar, però pot ser suficient en cas en alguns casos
  - per exemple, domini conegut on no és necessari entrar en detalls
  - e.g.: *El sistema ha de ser segur i fàcil d'usar*
- **quantitatiu**: millor per a validar
  - més fàcil si hi ha un sistema similar per a comparar-nos
  - e.g.: *El sistema ha de processar el pagament en menys de 2 segons*
- **operacionalitzat**: expressa un requisit de qualitat en funció de requisits funcionals
  - e.g.: *En fer login, el sistema ha de d'encriptar el nom d'usuari i contrasenya* (requisit de seguretat operacionalitzat)

Requisits de qualitat - classificació

Existeixen una gran quantitat de classificacions disponibles, e.g.:

- **ISO/IEC 25010**: estàndard que ofereix 8 característiques de qualitat descomposades en subcaracterístiques
  - *Functional Suitability*, *Reliability*, *Performance Efficiency*, *Security*, *Operability*, *Compatibility*, *Maintainability*, *Portability*
- **Volere**: es basa en una plantilla per descriure requisits
  - no només de qualitat
  - també 8 tipus de requisits de qualitat (alguns en realitat són restriccions, e.g. relacionats amb aspectes legals)

En qualsevol cas, la classificació és un ajut per identificar i gestionar els requisits

Restriccions

Existeixen diverses categories:

- **tècniques**: e.g., ús d'un cert protocol o component
  - e.g., treballar amb passarel.la de pagament de Caixabank
- **legals**: imposades per la llei, estàndards, …
  - e.g., compliment de la llei de protecció de dades europea (GDPR)
- **organitzacionals**: referents a processos o polítiques de l'organització
  - e.g., no permetre devolucions d'entrades comprades
- **culturals**: referents a la cultura on viuen els usuaris
- **d'entorn**: especialment en sistemes cíber-físics, fan referències a condicions ambientals (temperatura, …)
- i qualsevol altre restricció que les parts implicades considerin

Bibliografia

- M. Glinz, H. van Loenhoud, S. Staal, S. Bühne. *Handbook for the CPRE Foundation Level according to the IREB standard*, v1.1.0, 2022.
- Ian Sommerville. Integrated Requirements Engineering: A Tutorial. IEEE Softw. 22(1): 16-23 (2005)
- ISO/IEC/IEEE 29148:2018. Systems and software engineering — Life cycle processes — Requirements engineering. ISO standard organization (2018)

Recursos

- https://www.ireb.org/en

2.2 Model de l’especificació

Conceptes bàsics: Model

"a description or analogy used to help visualize something (such as an atom) that cannot be directly observed"

– diccionari Merrian-Webster

or a software system!

- donen un fonament sòlid a l'enginyeria (del software)
- en INEP, un model representa una *projecció* del problema a resoldre
  - projecció: etapa (e.g., anàlisi de requisits vs. disseny arquitectònic), focus (e.g., dades vs. comportament), …

Per què usar models?

- *comprensibilitat*: els elements del sistema i les seves interconnexions són més fàcils de comprendre
- *focus*: cada model es focalitza en una perspectiva i fa més senzill comprendre els requisits modelats
- *correctesa*: els models segueixen unes regles que ajuden a evitar ambigüitats i inconsistències
- *suport*: possibilities alguna forma de anàlisi i/o processat dels requisits

Per què NO usar models?

- *aprenentatge*: la tasca de modelatge requereix d'un temps d'aprenentatge
- *consistència*: sincronitzar els diversos models d'un sistema pot no ser trivial
- *capacitat expressiva*: és pràcticament impossible que un model abasti tots els aspectes d'un sistema
  - p.e., requisits de qualitat
- *aplicabilitat*: el modelatge no sempre encaixa bé en un procés de desenvolupament de software real

Aleshores, els usem? no els usem? només una mica?

Depèn:

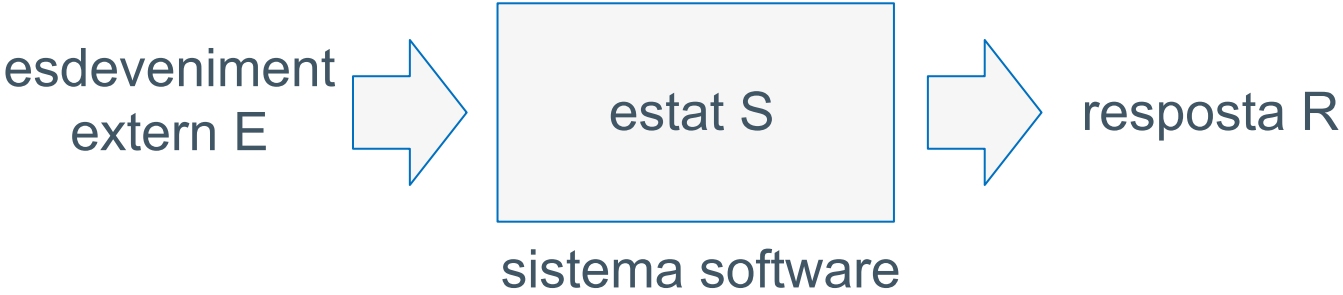
- cada context és un món
- convé estar preparat per al seu ús intensiu o només ocasional

Hipòtesi INEP: ús intensiu dóna millors resultats

Hipòtesi AMEP: ús ocasional dóna millors resultats

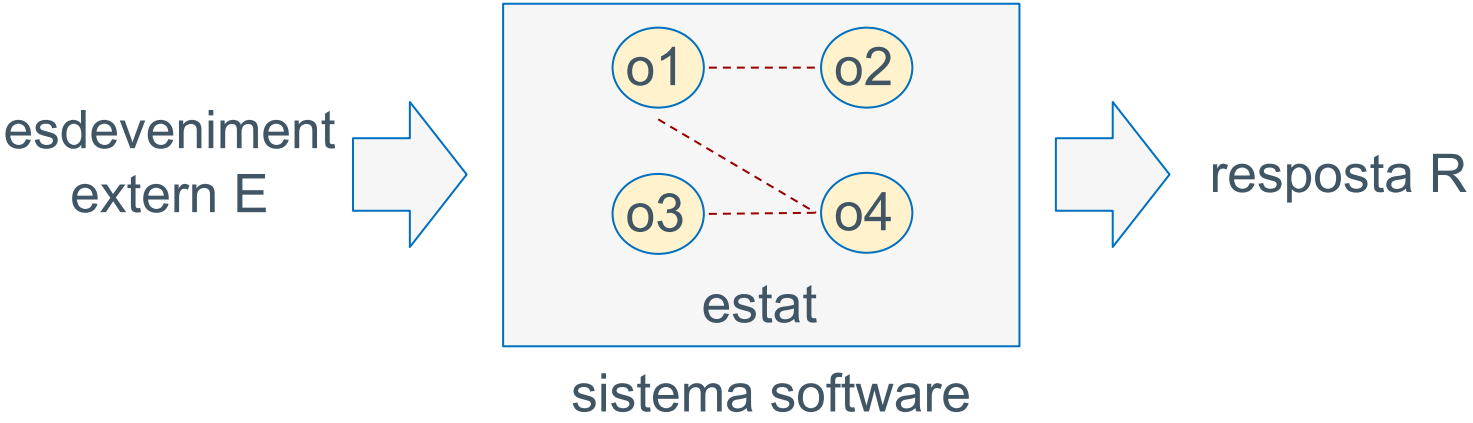
Cal buscar sempre el punt d'equilibri!

Models per a especificar un sistema software



- **model de dades**: conjunt de condicions que permeten discernir que S és un estat vàlid del sistema
- **model de comportament**: per tot esdeveniment extern E i estat S, especifica la resposta R del sistema
  - algunes respostes es produeixen sense E, típicament pel pas del temps

Models orientats a objectes (1)



el sistema software es conceptualitza com una col.lecció d'objectes que presenten relacions estructurals i que es comuniquen per assolir els requisits establerts

Models orientats a objectes (2)

Els models orientats a objectes (O.O.) s'apliquen tant a l'etapa d'especificació com a la de disseny:

- el model d'especificació expressa la conceptualització del problema
- l'etapa de disseny elabora aquest model d'especificació afegint-hi detall que l'apropa al món dels llenguatges de programació
  - bàsicament, els elements que formen ambdós models són els mateixos
  - de fet, la frontera entre model d'especificació i de disseny no sempre és nítida

A INEP, usarem models orientats a objectes

Conceptes bàsics: Diagrama

"a graphic design that explains rather than represents"
especially: "a drawing that shows arrangement and relations (as of parts)"

– diccionari Merrian-Webster

- a INEP, usarem diversos tipus de diagrames amb diferents propòsits
- els diagrames es dibuixen seguint les regles d'un cert llenguatge de modelització (UML en el nostre cas)

Conceptes bàsics: Llenguatges de modelització

Consisteixen en:

- sintaxi: conjunt de regles gramaticals
- semàntica: significat de cada constructor del llenguatge

Categorització:

- informal: basat en llenguatge natural
- semi-formal: normalment, notació gràfica
- formal: permet verificar i/o validar models

Combinable:

- e.g., llenguatge semi-formal amb annotacions textuals

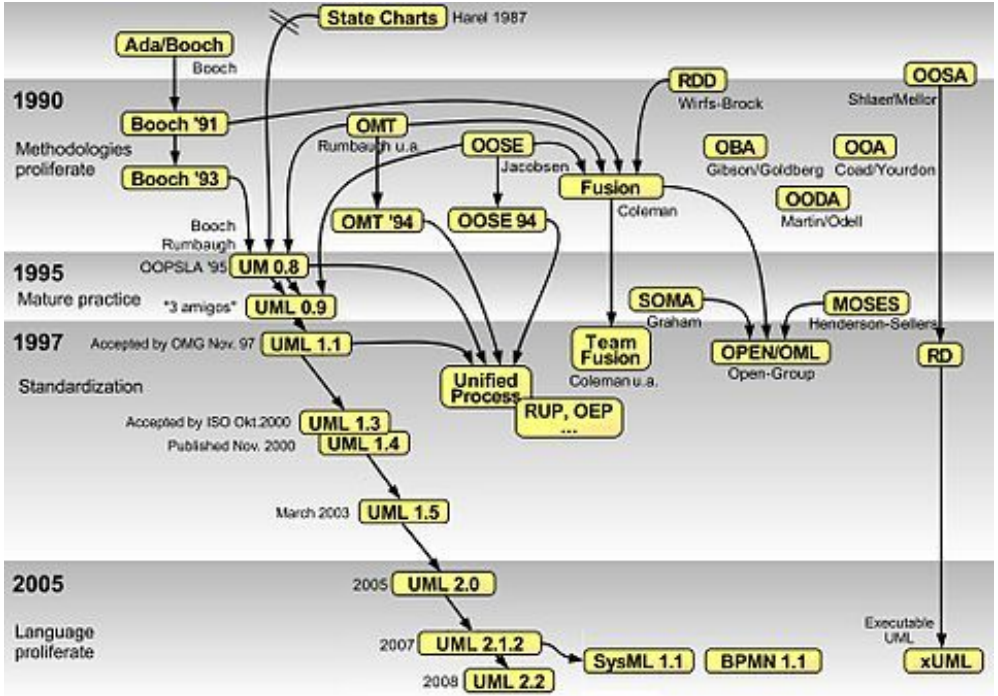
El llenguatge de modelització UML (1)

UML: Unified Modeling Language

Dissenyat primordialment per a la modelització O.O.

Actualment, versió 2.5

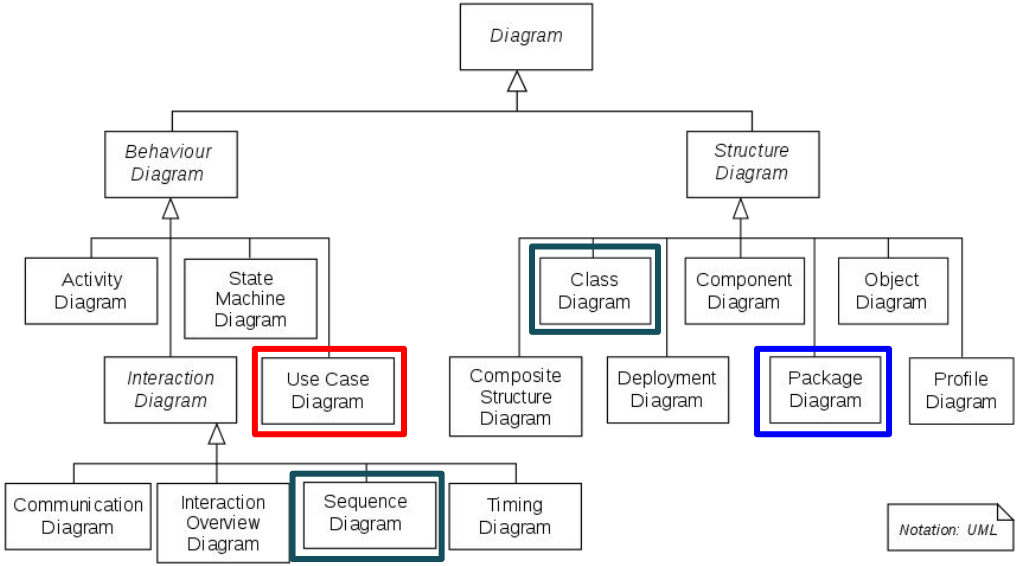
Relacions amb (R)UP, OCL, SysML, …



https://en.wikipedia.org/wiki/Unified_Modeling_Language

El llenguatge UML (2)

UML consta de diversos models, representats mitjançant diagrames acompanyats de text



: només a especificació

: només a disseny

: a especificació i disseny

https://en.wikipedia.org/wiki/Unified_Modeling_Language

El llenguatge UML (3)

A la part d'especificació, usarem els següents diagrames:

- **diagrama de casos d'ús**: per representar la funcionalitat del sistema
  - descripció textual dels casos d'ús individuals
- **diagrama de classes**: per representar el model de dades
  - restriccions d'integritat textuals
- **diagrama de seqüència**: per representar el flux de dades i operacions
  - descripció textual de contractes d'operacions

Resum de conceptes: general

món real

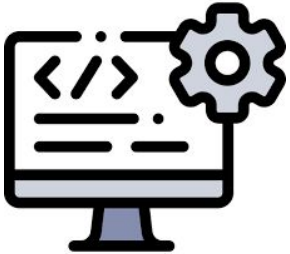
projecció

conceptualització

$f(x)$

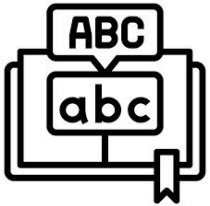
model(s)

implementació

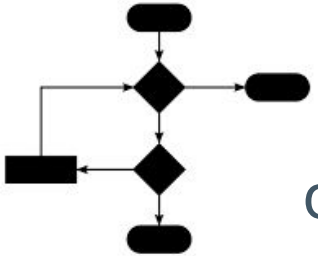


sistema software

representació



llenguatge

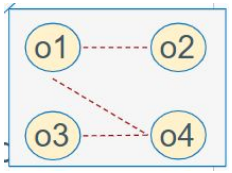


diagrama

Resum de conceptes: INEP

món real

projecció



model O.O.

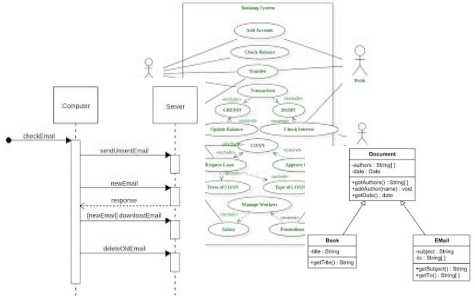
implementació

programa C++

conceptualització

representació

UML



diagrames UML

2.2.1 Model de casos d’ús

Diagrames de casos d’ús i descripcions individuals

Motivació

Els requisits resultat de la fase d'elicitació són entitats atòmiques individuals, però no reflecteixen:

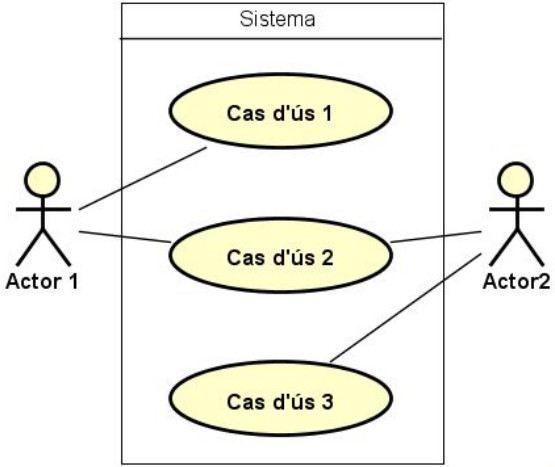
- la seva agrupació lògica
- la forma en què el sistema els organitza en funcionalitats accessibles pels usuaris
  - e.g., opcions de menú
- el flux d'interaccions usuari-sistema dins d'aquestes funcionalitats

El model de casos d'ús té com a objectiu organitzar els requisits afegint-hi aquesta informació

Model de casos d'ús

Compost per dos elements:

- **diagrama de casos d'ús** (en UML)
- **descripció individual dels casos d'ús** (en llenguatge natural)




```
cas d´ús cas d'ús 1
      escenari principal descripció
      escenaris alternatius
            alternativa-1: descripció
            alternativa-2: descripció
```

Actors

Entitat que interacciona amb el sistema en algun moment [*]

Tipologia:

- humans (subconjunt de les parts implicades)
  - e.g., client, organitzador de concert, administrador de sistema
- altre sistema software amb què el nostre sistema es comunica
  - e.g., pasarel.la de pagament d'un banc
- organitzacions externes
  - e.g., SGAE
- dispositius físics
  - e.g., sensor de presència

[*] Terminologia pròpia del model de casos d'ús. En altres contexts, el terme *actor* pot tenir un significat diferent

Escenaris

Seqüència d’esdeveniments que ocurreixen quan el sistema executa certa funcionalitat

Dos tipus d’esdeveniments:

- interaccions entre uns actors i el sistema
  - esdeveniment extern: interacció iniciada per un actor
- accions abans o després d’aquestes interaccions

Dos tipus d’escenaris:

- **escenari principal**: flux habitual d’esdeveniments
- **escenaris alternatius**: identifiquen situacions no tan habituals i descriuen el flux en aquests casos

Escenaris: exemple

Cas d’ús per enregistrar la compra d’entrades en un concert:

- *escenari principal*: El Sistema mostra la llista de concerts amb entrades disponibles. L’Usuari n’elegeix un. El Sistema mostra tota la informació disponible del grup o músic que toca en el concert i demana a l’Usuari que entri el número d’entrades que vol comprar (màxim 10). L’Usuari entra el número desitjat i, en resposta, el Sistema mostra el preu total. L’Usuari entra número, data d’expiració i CCV d’una targeta de crèdit. El Sistema es connecta amb la Pasarel.la de Pagament de la tarja i en delega el cobrament. Un cop Sistema rep confirmació del pagament, envia un email a l’Usuari amb el resum de la compra.
- *un escenari alternatiu*: el número d’entrades demanades per l’Usuari ultrapassa el número d’entrades disponibles: l’Usuari o bé entra un número d’entrades menor que aquest màxim, o bé abandona la compra

Context del sistema

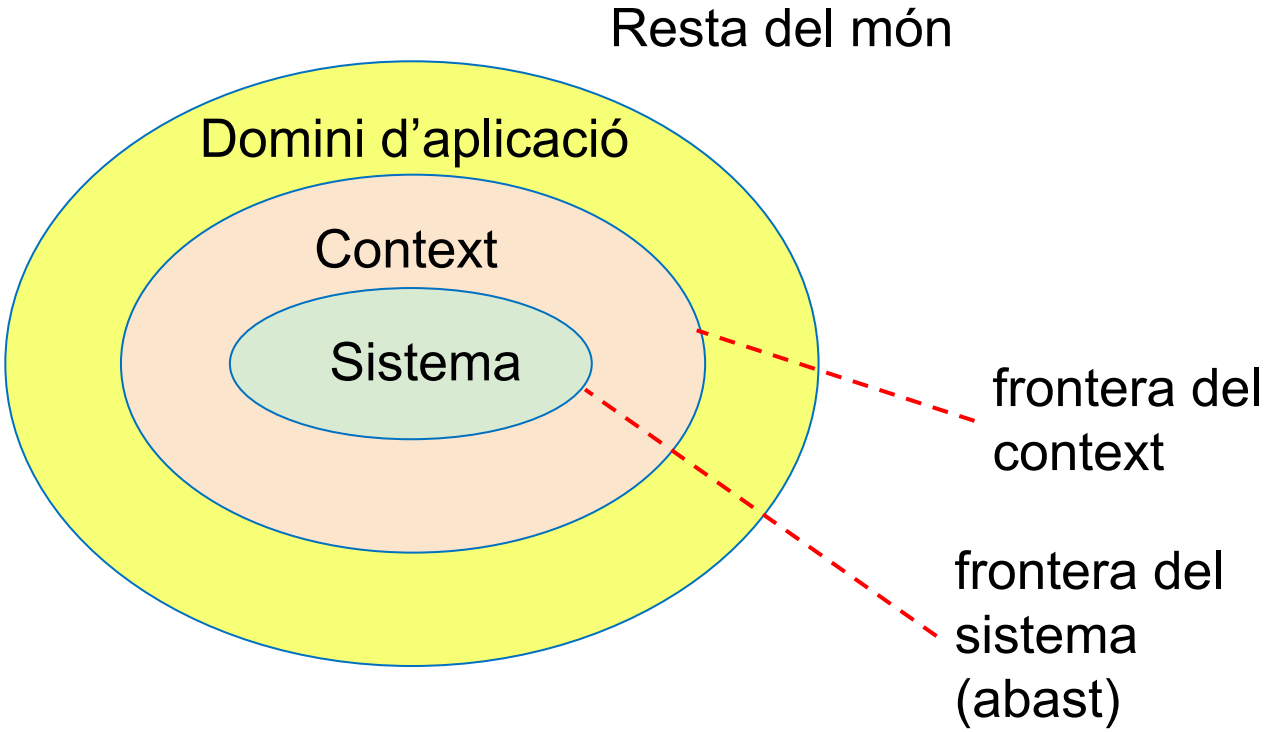
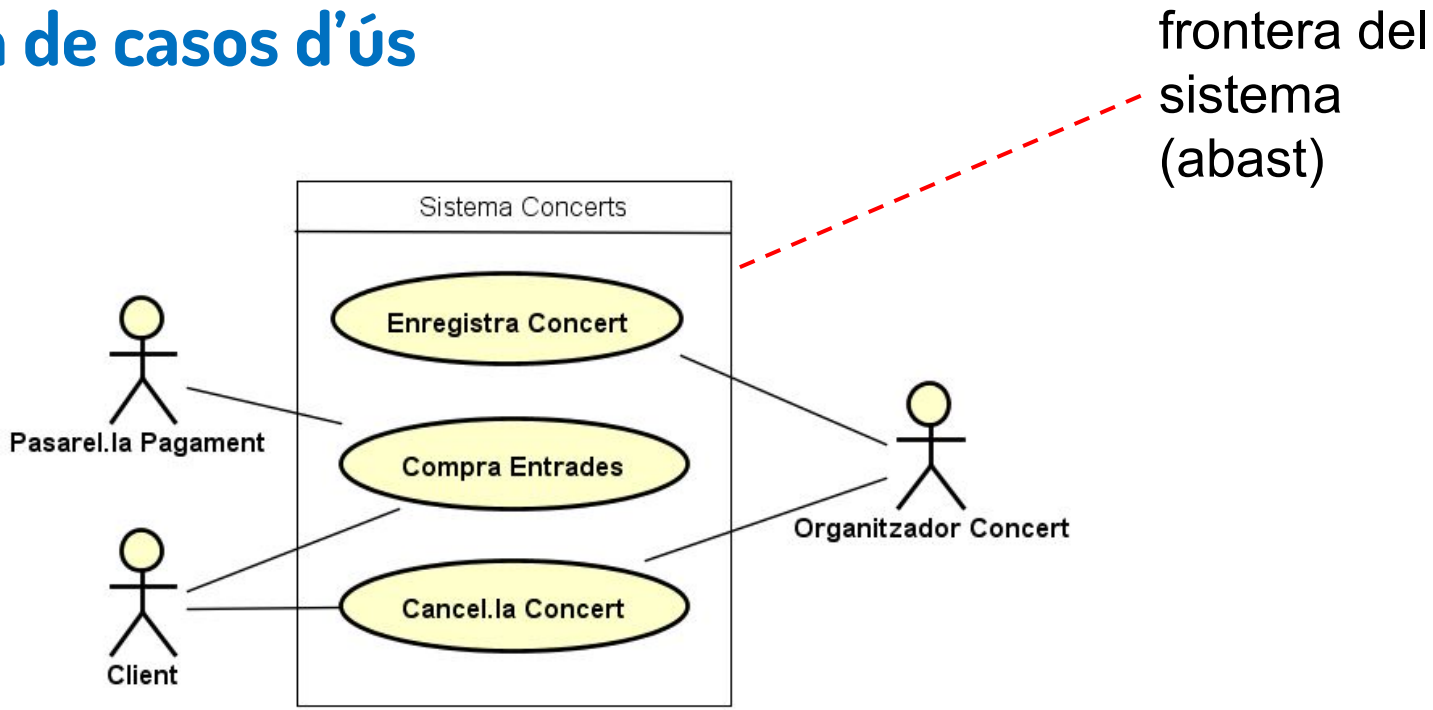


Diagrama de casos d'ús

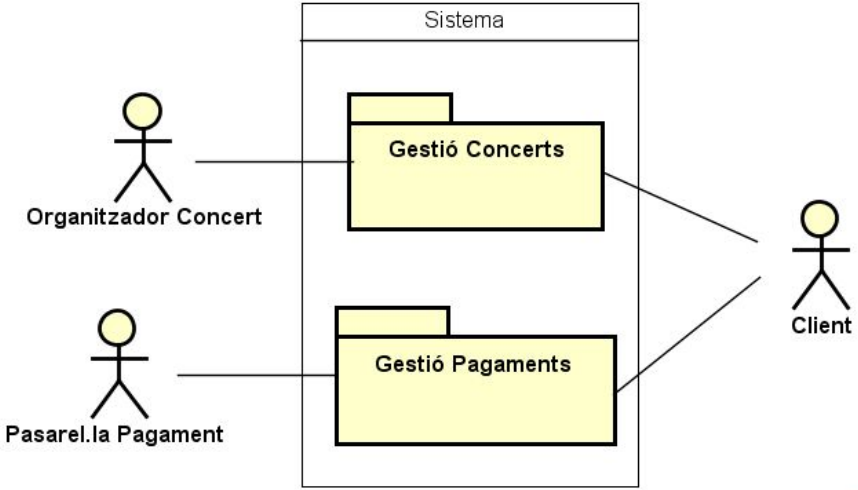


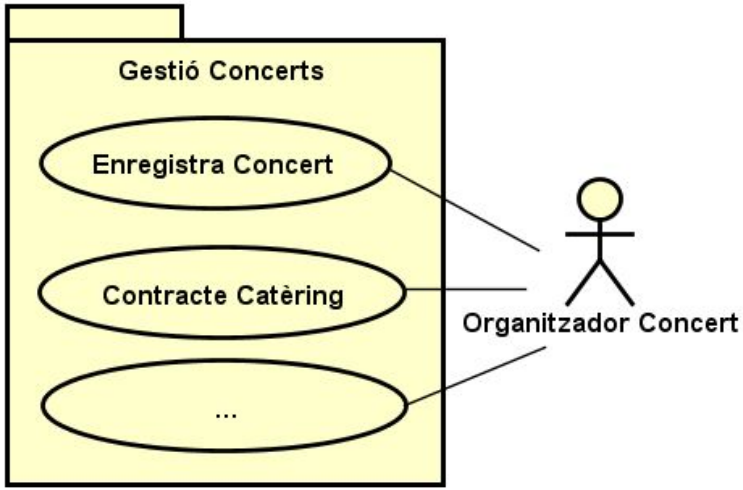
UML inclou construccions addicionals (jerarquies d'actors, relacions entre casos d'ús) que no usarem a l'assignatura amb una excepció…

Estructuració del diagrama de casos d'ús

Si el sistema conté dotzenes o centenars de casos d'ús, és útil usar el concepte de paquet, que ens permet agrupar casos d'ús "similars"

Cada paquet s'especifica al seu torn amb un diagrama propi





Descripció individual dels casos d'ús

Diversos formats de descripció:

- **abreujat**: només inclou una descripció abreujada, que es focalitza en els aspectes més rellevants
- **detallada**: proporciona tota mena de detalls relatius als escenaris, actors, dades, etc.

En tots dos casos, és essencial definir una plantilla de treball

A INEP, usarem l'estil abreujat

↓

El model del comportament proporcionarà més detalls

Plantilla de descripció abreujada

```
cas d'ús <nom del cas d'ús>
    activació <condició d'inici>
    escenari principal <descripció>
    escenaris alternatius
        <condició 1>: <descripció 1>
        <condició 2>: <descripció 2>
        …
    requisits no funcionals
        <requisit de qualitat / restricció 1>
        <requisit de qualitat / restricció 2>
        …
```

Exemple

<u>cas d'ús</u> Compra Entrades

<u>activació</u> L'Usuari decideix comprar entrades per a un concert

<u>escenari principal</u> v. Transpa 48

<u>escenaris alternatius</u>

el número d'entrades demanades per l'Usuari ultrapassa el número d'entrades disponibles: v. Transpa 48

la Pasarel.la de Pagament rebutja la transacció: el Sistema dóna l'opció de tornar a provar, fins a un màxim de 3 cops

etc...

<u>requisits no funcionals</u>

El Sistema ha d'assegurar que les dades de la tarja de crèdit no queden exposades en cap moment

2.2.2 Model de dades

Diagrama de classes

Visió del món real com a col.lecció d'objectes (1)

Visió del món real com a col.lecció d'objectes (2)

toca

THE BEATLES

són membres

Objecte

Entitat física o lògica que existeix en el món real:

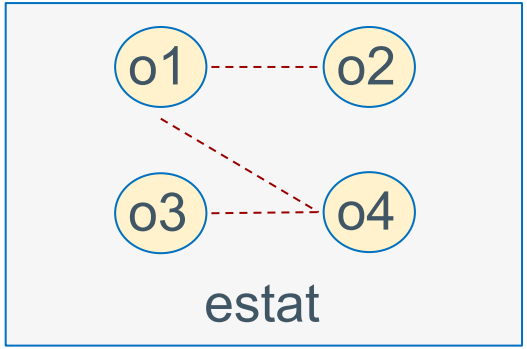
- física: un músic (Ringo), un instrument (bateria)
- lògica: un grup musical (Beatles), un concert (4/5/68)

Els objectes es creen, es poden modificar i eventualment es poden eliminar

Els objectes estan relacionats:

- un músic és *membre* d'un grup musical
- un grup musical *dóna* un concert

Estat d'un sistema des de la perspectiva orientada a objectes



sistema software

Internament, cada objecte s'identifica mitjançant un identificador únic que anomenem ***oid*** (inicials en anglès)

Models de dades orientats a objectes

El model de dades d’un sistema descriu el seu estat

Consisteix en la col.lecció d’objectes que emmagatzema, i les seves relacions

Els diagrames corresponents han de permetre descriure aquests objectes i relacions de forma semànticament rica

UML adopta aquesta perspectiva, i ofereix els **diagrames de classe** per representar els models orientats a objectes

Atribut

Denota una propietat *rellevant* d'un objecte

- rellevant: és d'interès per al sistema en desenvolupament

- nom
- edat
- número de discos venuts (en milions)
- ~~número de peu que calça~~

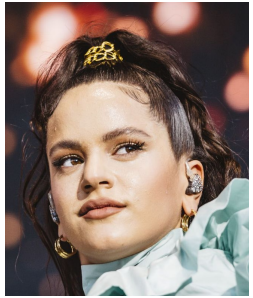
Classe

Caracteritza una col.lecció d'objectes *similars*:

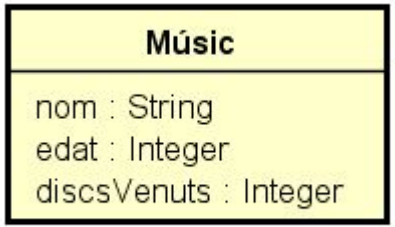
- tenen els mateixos atributs
- tenen les mateixes relacions amb objectes d'altres classes
- es comporten de la mateixa manera

John (oid = 123)

Rosalia (oid = 456)



objectes **instància** de la classe `Music`



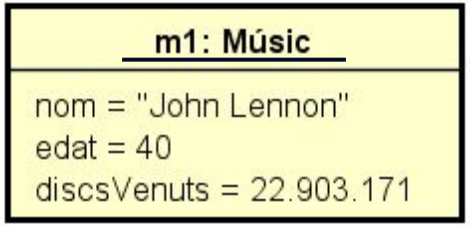
classe `Music` en UML

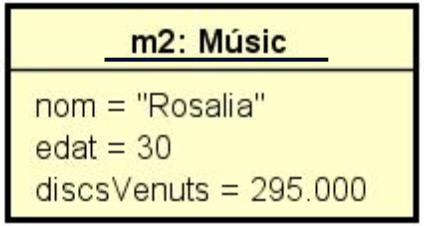
Classes i objectes

Un objecte <u>sempre</u> és instància d'alguna classe

Representació:

- UML normatiu: diagrames d'objectes





- més pràctic: representació textual

```
                    Music
(#m1 , "John Lennon", 40, 22M)
(#m2 , "Rosalia", 30, 3M)
```

no cal si es veu clar

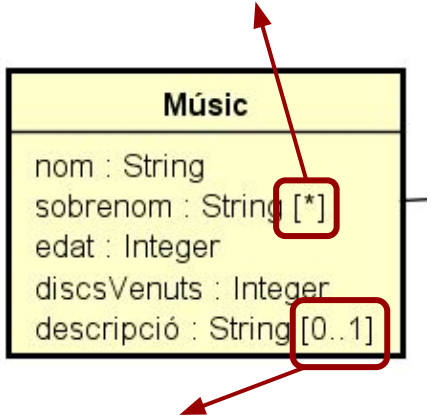
oids

Propietats dels atributs - multiplicitat

Per defecte, els atributs d'un objecte tenen un **valor** d'un cert **tipus**

De vegades:

- els atributs poden no tenir valor (*amb valor nul*)
- els atributs poden tenir més d'un valor (*multi-valuat*)



Propietats dels atributs - claus (1)

Com distingir dues instàncies d'una mateixa classe?

- oid: identificador intern, assignat pel sistema
  - en l'estat d'un sistema software, no poden haver-hi dues instàncies *de qualsevol classe* amb el mateix valor d'oid
- **clau**: identificador lògic, assignat per l'enginyer
  - en l'estat d'un sistema software, no poden haver-hi dues instàncies *de la mateixa classe* amb el mateix valor de clau

Cap objecte no pot tenir valor nul a la clau

Propietats dels atributs - claus (2)

Quina forma pren la clau?

- un únic atribut clau (cas habitual)

Músic

nom : String {id}
sobrenom : String [*]
edat : Integer
discsVenuts : Integer
descripció : String [0..1]

- clau multi-atribut

(#m1, "John Lennon", 40, …)
(#m2, "Rosalia", 30, …)

subratllades

Concert

lloc : String {id}
data : Date {id}
pvp : Float

(#c1, "Sant Jordi", 1/1/24, 30)
(#c2, "Nou Camp", 1/8/24, 50)

- clau dèbil (queda pendent…)

Exemple: classe Ciutat

Teniu dos minuts per pensar com podria ser una classe que modeli les ciutats en un sistema d'informació geogràfic

Propostes?

Exemple: classe Ciutat

Quina proposta és la correcta?

Segurament, totes!!

Sense informació del problema (p.e., en forma de requisits), no ho podem saber!!

Exemple: classe Ciutat

Considereu ara els requisits següents:

R1. El sistema ha de guardar el nom i número d'habitants de les ciutats

R2. El sistema ha de permetre identificar quines ciutats són capital de província

R3. Per aquelles ciutats que són capital de província, el sistema ha de dir també des de quin any ho són

Com queda la classe?

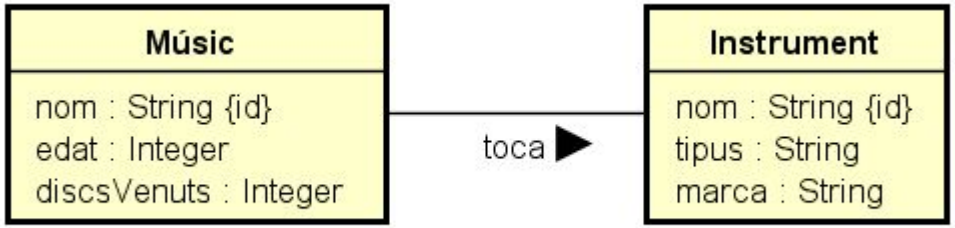
Associacions

Representació de les relacions que existeixen al món real

- s’estableixen entre classes
- s’instancien entre objectes

Associacions

Nivell classe:

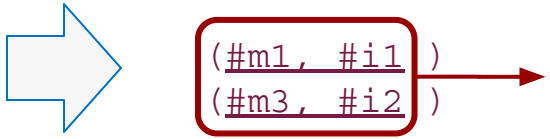


Nivell objecte:

```
(#m1, "John Lennon", 40, 22M)
(#m2, "Rosalia", 30, 3M)
(#m3, "Ringo Starr", 83, 2M)
```

```
(#i1, "guitarra", "corda", "Rickenbacker")
(#i2, "bateria", "percussió", "Ludwig")
```

John Lennon toca una Rickerbacker
Ringo Starr toca una Ludwig



clau de l'associació

Les associacions no poden tenir un dels components amb valor nul

Exemple: associació per enregistrar on viuen les persones (1)

Teniu dos minuts per pensar com podria ser una associació que relacioni persones amb ciutats segons els requisits:

R1. El sistema ha d'enregistrar en quina ciutat viu cada persona

R2. Una persona pot viure en més d'una ciutat

Propostes?

Exemple: associació per enregistrar on viuen les persones (2)

Considereu ara els requisits següents:

R1. El sistema ha d'enregistrar en quina ciutat viu cada persona

R2. Hi ha persones que no tenen domicili conegut

R3. En cas que una persona visqui en més d'una ciutat, volem saber la més recent

Hi ha canvis?

Multiplicitats d'associacions

Permet afegir més precisió a les associacions

- cada instància de classe a l'extrem d'una associació, amb quantes instàncies de classe de l'altre extrem es pot associar com a mínim i com a màxim?

Respostes habituals:

- exactament una (multiplicitat 1)
- amb una o cap (multiplicitat 0..1)
- amb moltes (multiplicitat 0..* o més abreujadament, *)

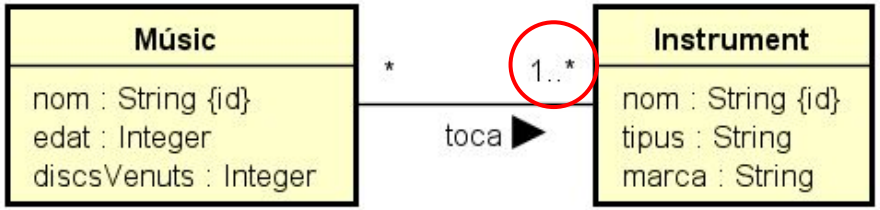
Però qualsevol altra resposta és lícita!

Exemple: associació per enregistrar on viuen les persones (3)

Penseu quina és la multiplicitat de les associacions en cadascuna de les dues situacions descrites

Hi ha diferències?

Multiplicitats d'associacions: coneixement del domini (1)



Paul McCartney normalment toca el baix, però de vegades també el piano (multiplicitat * per dalt)

Músic

(#m2, "Rosalia", 30, 2M)
(#m4, "Paul McCartney", 81, 45M)

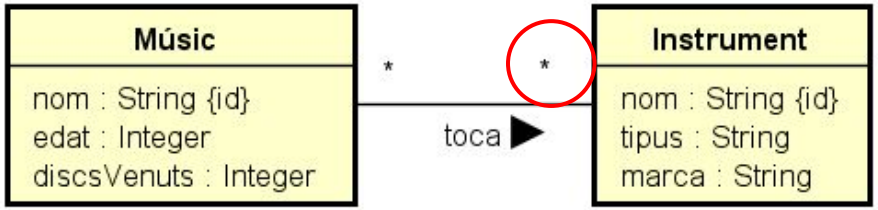
Instrument

(#i3, "baix", "corda", "Höfner")
(#i4, "piano", "percussió", "Bechstein")

toca

(#m4, #i3)
(#m4, #i4)
(#m2, ???) ← Què passa amb Rosalia?

Multiplicitats d'associacions: coneixement del domini (2)



Rosalia només canta (multiplicitat 0 per baix)

Músic

(#m2, "Rosalia", 30, 2M)
(#m4, "Paul McCartney", 81, 45M)

Instrument

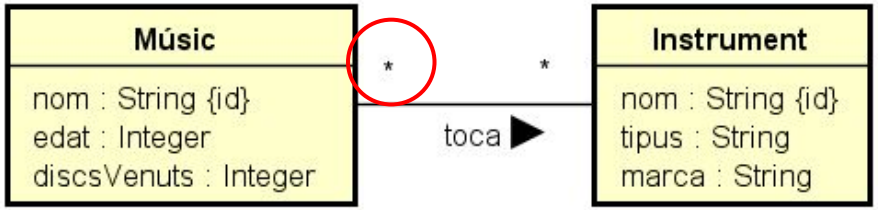
(#i3, "baix", "corda", "Höfner")
(#i4, "piano", "percussió", "Bechstein")

toca

(#m4, #i3)
(#m4, #i4)
~~(#m2, ???)~~ ← permès!

Multiplicitats d'associacions: coneixement de la solució

Tot i que a llarg termini, segurament tot instrument serà tocat per algun músic, això no serà així en els moments inicials de l'existència del sistema



Músic

```
#m4, "Paul McCartney", 81, 45M)
(#m5, "Jaco Pastorius", 35, 4M)
```

Instrument

```
(#i3, "baix", "corda", "Höfner")
(#i4, "piano", "percussió", "Bechstein")
```

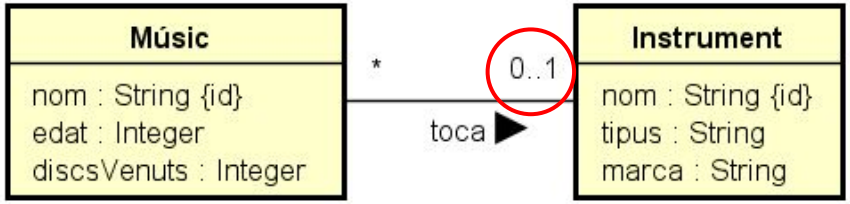
toca

```
(#m4, #i3)
(#m5, #i3)
```

estat vàlid del sistema

Multiplicitats d'associacions: importància dels requisits

Req: en cas que un músic toqui diversos instruments, ens quedem amb el més representatiu



(#m2, "Rosalia", 30, 2M)
(#m4, "Paul McCartney", 81, 45M)

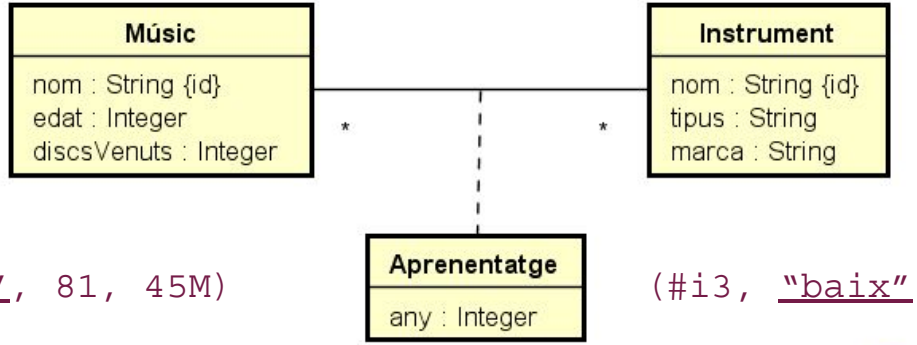
(#i3, "baix", "corda", "Höfner")
(#i4, "piano", "percussió", "Bechstein")

toca
(#m4, #i3)
~~(#m4, #i4)~~

Classe associativa

Tipus d'associació que té informació associada

Req: volem enregistrar l'any en què un músic va començar a tocar un instrument



(#m4, "Paul McCartney", 81, 45M)

(#i3, "baix", "corda", "Höfner")

les classes associatives tenen oid

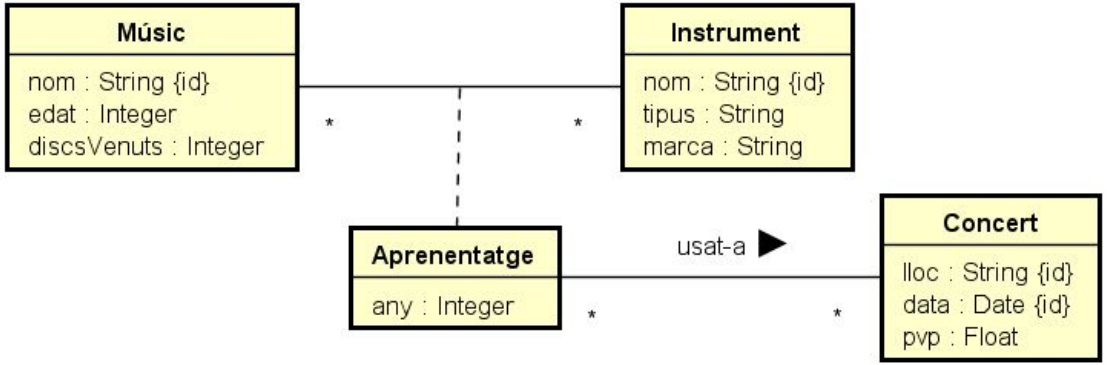
(#t3, #m4, #i3, 1960)

Classes associatives

Les classes associatives són associacions i classes

- com a classes, es poden associar amb altres classes

Req: enregistrar en quins concerts un músic va tocar un instrument donat



Aprenentatge(#t3, #m4, #i3, 1960)

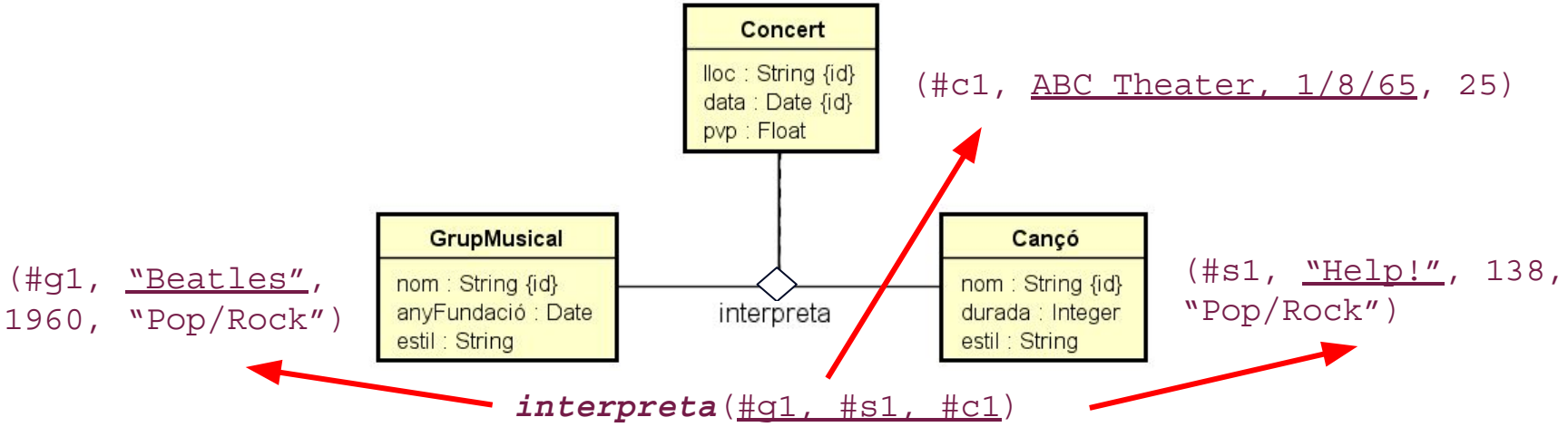
Concert(#c1, Apolo, 1/1/20, 25)

usat-a(#t3, #c1)

Associació ternària

Una associació ternària connecta objectes que són instàncies de tres classes diferents

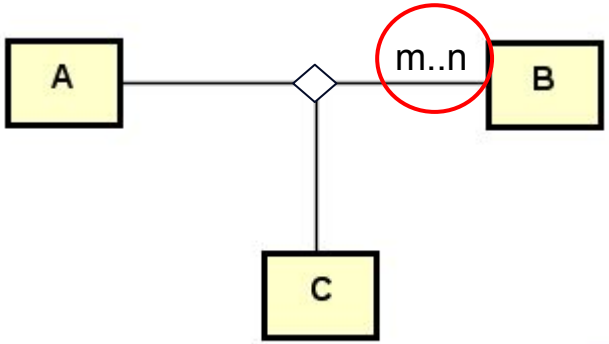
- en realitat, podria haver-hi associacions *n*-àries, però rarament *n* > 3



També poden ser associatives (exemple?)

Multiplicitats en les classes ternàries

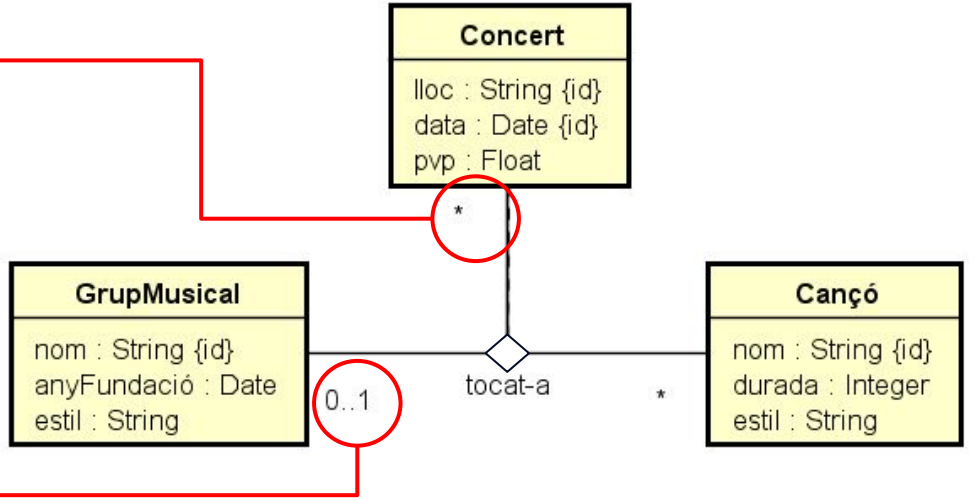
No és trivial…



- multiplicitat *m*: donada una parella possible d'objectes de classe A i C, com a mínim quantes instàncies de B **han** d'estar-hi associades?
- multiplicitat *n*: donada una parella d'objectes de classe A i C, com a màxim quantes instàncies de B **poden** estar-hi associades?

Multiplicitats en les classes ternàries: exemple

Un grup musical pot tocar la mateixa cançó a un nombre indeterminat de concerts

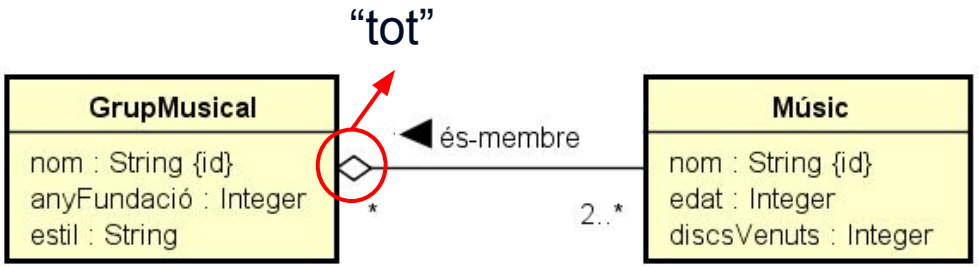


Una cançó donada, en un concert donat, només la pot tocar un grup musical com a màxim

Altres tipus d'associacions: Agregació

Defineix una relació tot-part

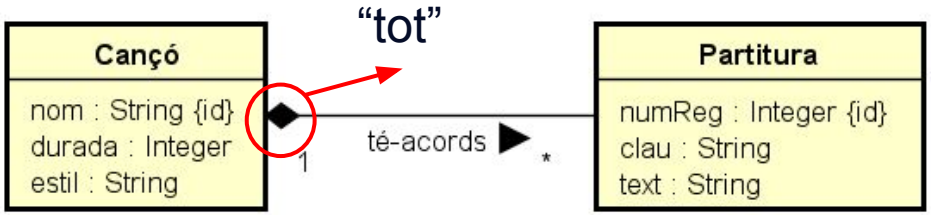
Les instàncies de la classe "tot" (composats) agreguen instàncies de la classe "part" (components)



Altres tipus d'associacions: Composició

Agregació en què l'eliminació d'un composat implica l'eliminació automàtica dels seus components

- per això, la multiplicitat del "tot" no pot ser més gran que 1



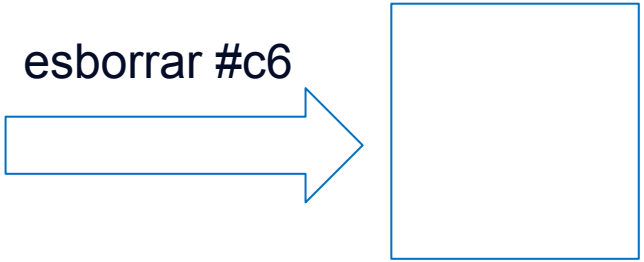
estat del sistema

Cançó(#c6, "YMCA", 226, "disco")

Partitura(#p3, 724391, "sol", "🎵🎶")

té-acords(#c6, #p3)

esborrar #c6

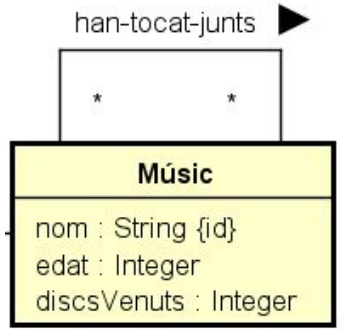


Altres tipus d'associacions: Associació recursiva

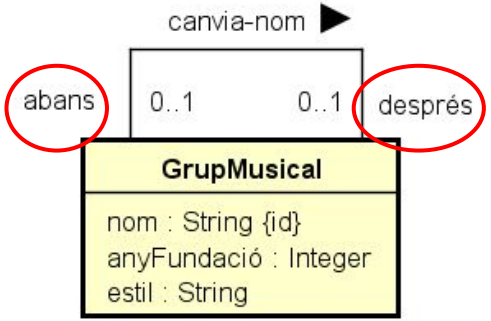
Associació on tots dos extrems són de la mateixa classe

Poden ser simètriques o no

- si no ho són, podem donar nom als seus extrems
  - de fet, això ho podem fer en qualsevol associació



simètrica

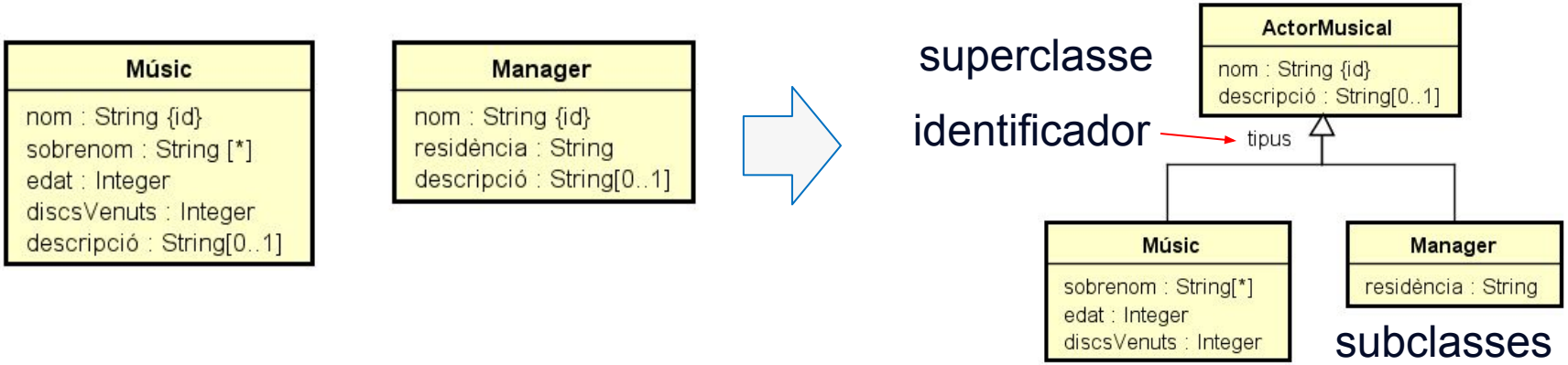


no simètrica

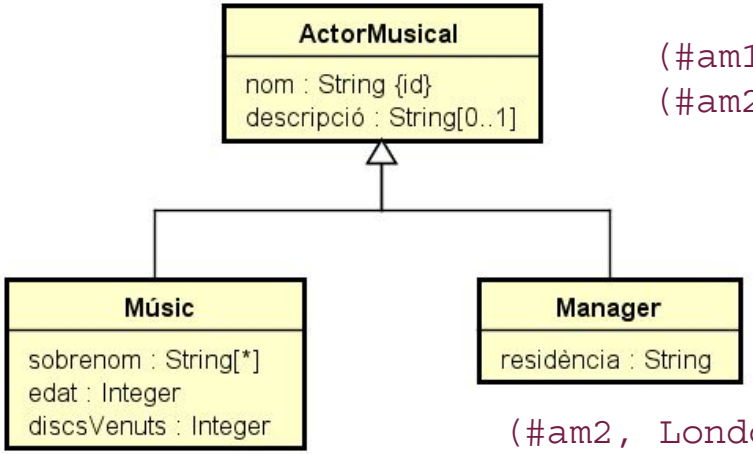
Especialització

Mecanisme que permet definir una classe (**subclasse** o *filla*) afegint detall sobre una altra (**superclasse** o *pare*) resultant en una **jerarquia** de classes

Es pot considerar al revés i es diu **generalització**



Especialització a nivell d'objectes. Herència



(#am1, "Tom Waits", -)
(#am2, "Brian Epstein", -)

(#am1, -, 73, 4M)

(#am2, London)

Totes les instàncies de les subclasses ho són també de la superclasse

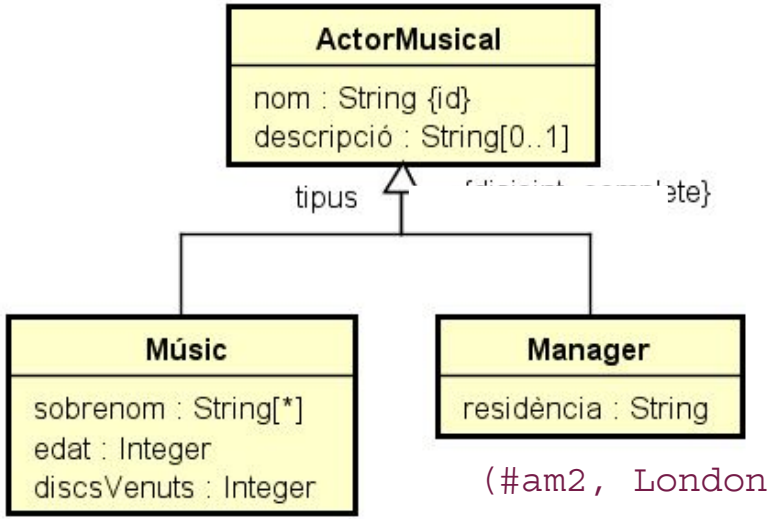
Les instàncies de les subclasses **hereten** els valors dels atributs i les associacions del seu pare

(#am1, "Tom Waits", -, 73, 4M, -)

Qualificadors de les especialitzacions

(#am1, "Tom Waits", -, Músic)
(#am2, "Brian Epstein", -, Manager)
(#am3, "J.J. Cale", "Soft", Músic)

impossible! (complete)



(#am2, London)

(#am1, -, 73, 4M)
(#am2, -, 33, 10M)

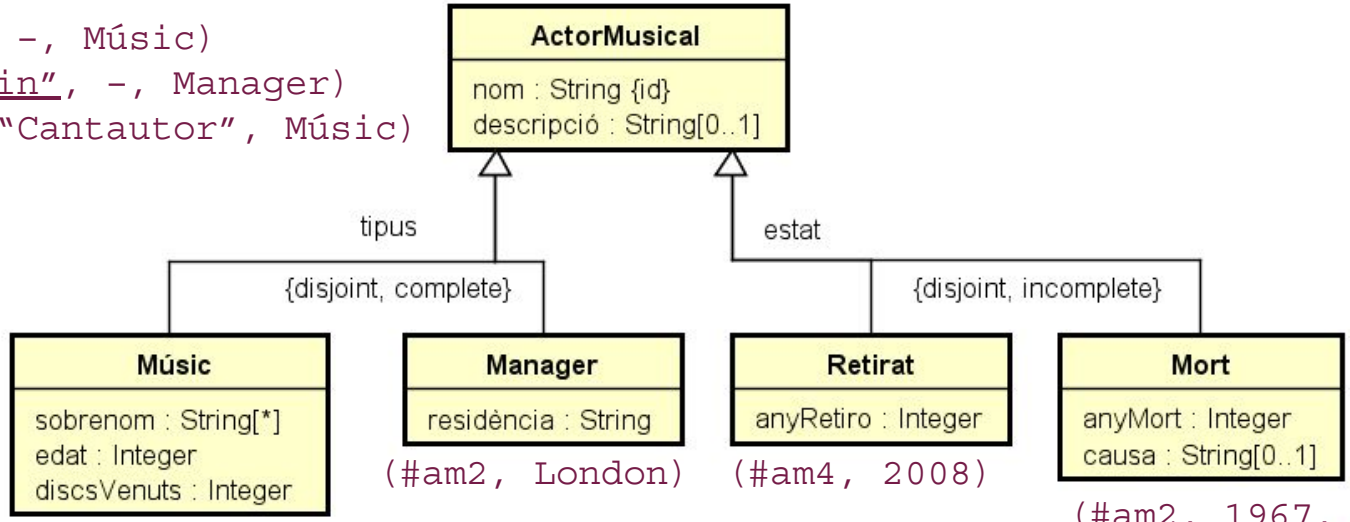
impossible! (disjoint)

- una instància de superclasse, pot no ser instància de subclasse?
  - Sí: *incomplete*; No: *complete*
- una instància de superclasse, pot ser instància de més d'una subclasse?
  - Sí: *overlapping*; No: *disjoint*

Casos particulars: múltiples jerarquies

Superclasse que admet diversos conceptes d'especialització

(#am1, "Tom Waits", -, Músic)
(#am2, "Brian Epstein", -, Manager)
(#am4, "en Llach", "Cantautor", Músic)



(#am1, -, 73, 4M)
(#am4, -, 33, 1M)

(#am2, London)

(#am4, 2008)

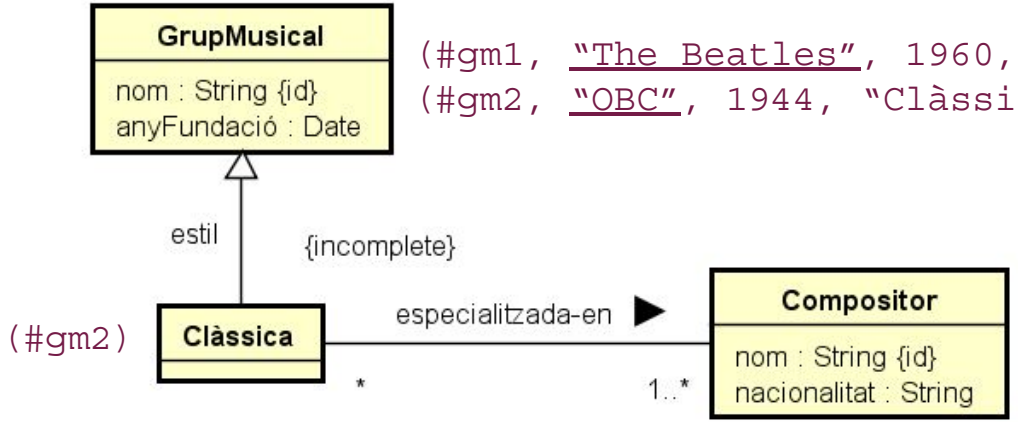
(#am2, 1967, suïcidi)

Informació completa d'en Llach:

(#am4, "en Llach", "Cantautor", Músic, -, 33, 1M, Retirat, 2008)

Casos particulars: superclasses amb una única subclasse

Permet afegir informació en alguns casos concrets



Qualificació:

- sempre *incomplete*
- no cal dir *overlapping / disjoint* (no té sentit)

Altres conceptes: Restriccions d'integritat textuals (1)

El diagrama de classes inclou restriccions d'integritat gràfiques que restringeixen els estats vàlids del sistema, p.e:

- qualificadors a les especialitzacions
- multiplicitat a les associacions

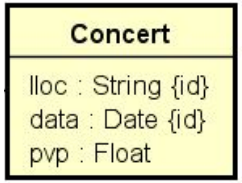
Però hi ha altres restriccions d'integritat que no es poden expressar gràficament en UML

- s'afegeixen com a text complementari

UML ofereix el llenguatge OCL, però com que pot ser feixuc, nosaltres les escriurem en llenguatge natural

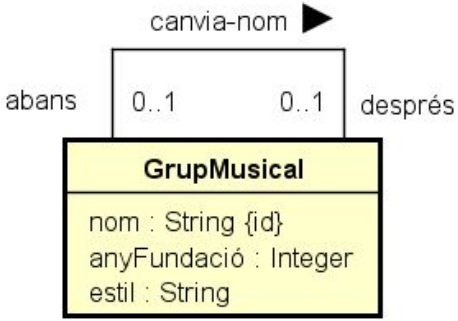
Altres conceptes: Restriccions d'integritat textuals (2)

Cas típic 1: restricció dels valors d'un atribut




```
RIT1(c: Concert): c.pvp ≥ 0.0
```

Cas típic 2: relació entre els valors de dos atributs




```
RIT2(gm1, gm2: GrupMusical): si gm1 va canviar el nom
de gm2, aleshores es compleix que gm1.anyFundació ≥
gm2.anyFundació
```

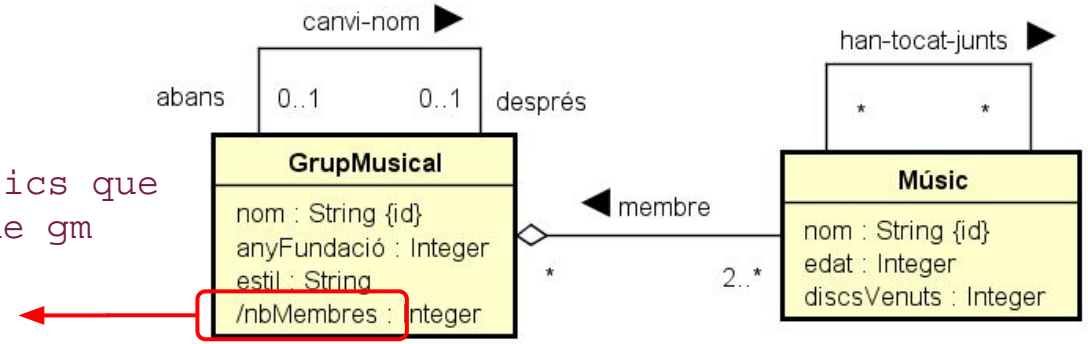
Altres conceptes: informació derivada

Es poden incloure en el diagrama atributs i associacions els valors dels quals són calculables a partir d'altres elements:

- normalment, per comprensibilitat o comoditat
- el seu valor es defineix també textualment

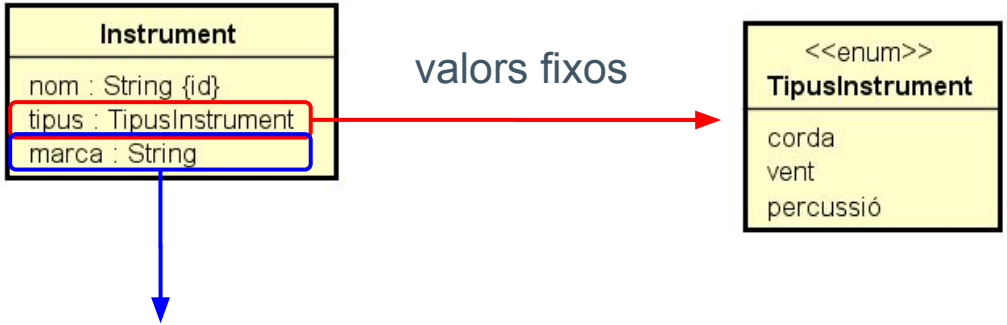
```
ID1(gm: GrupMusical):
  gm.nbMembres = número de Músics que
                 són membres de gm
```

"/": símbol d'informació derivada



Altres conceptes: enumeracions

Alternativa al tipus String quan els valors possibles són predeterminats



valors desconeguts a priori i variables

Altres conceptes: classes úniques ("singleton")

Classes que admeten un únic objecte com a instància

Permeten incloure en el model informació rellevant en forma d'atributs

Exemple: imaginem que estem desenvolupant el nostre sistema musical per a una companyia anomenada "La Traviata":

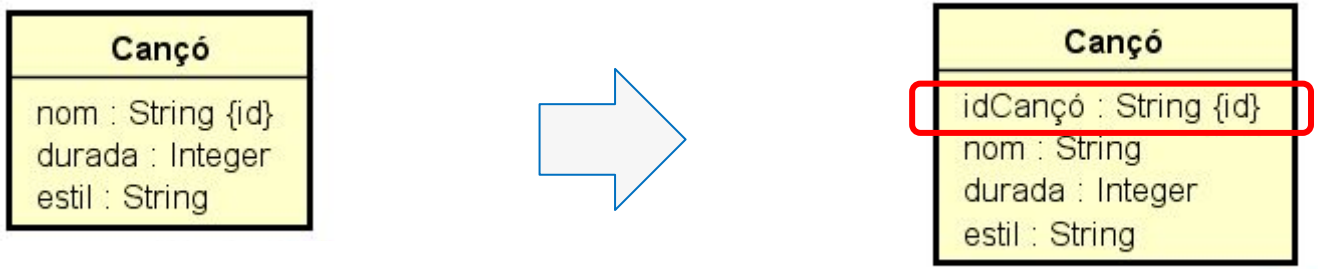
multiplicitat de classe (en realitat, pot ser qualsevol valor)

```
Companyia(#cp1, "La Traviata", "c/Verdi 89", 34)
```

Altres conceptes: clau substituta (*surrogate*)

Clau introduïda artificialment amb l'única finalitat de ser capaços de distingir instàncies d'una classe

- quan cap atribut (o conjunt d'atributs) de la classe identifica únicament els objectes
- el sistema haurà de generar-les automàticament



Cançó

(#c1, ~~"The End"~~, 701, "Psycho rock")
(#c2, ~~"The End"~~, 140, "Pop rock")

Cançó

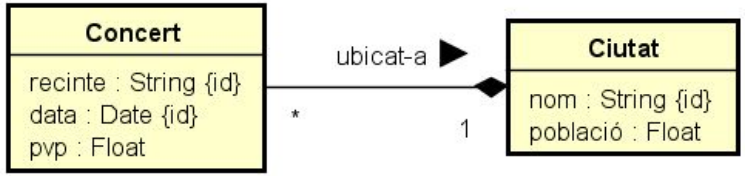
(#c1, "S197Doo", "The End", 701, "Psycho rock")
(#c2, "S323Bea", "The End", 140, "Pop rock")

Altres conceptes: clau dèbil (1)

(Part de) la clau d'una classe es troba en una altra classe

- les classes estan unides per algun tipus d'associació amb multiplicitat 1 a l'extrem corresponent a la classe amb clau

(#c1, "Platja", 1/8/23, 15)
(#c2, "Platja", 1/8/23, 30)



(#t1, "BCN", 2M)
(#t2, "Madrid", 3M)

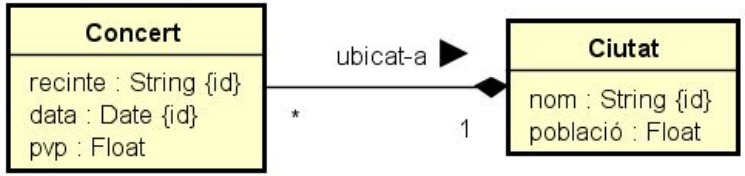
(#c1, #t1)
(#c2, #t2)

Altres conceptes: clau dèbil (2)

Problema notacional: com representar-lo?

- no és possible en UML 2.5
- cal afegir restricció d'integritat textual

(#c1, "Platja", 1/8/23, 15)
(#c2, "Platja", 1/8/23, 30)



(#t1, "BCN", 2M)
(#t2, "Madrid", 3M)

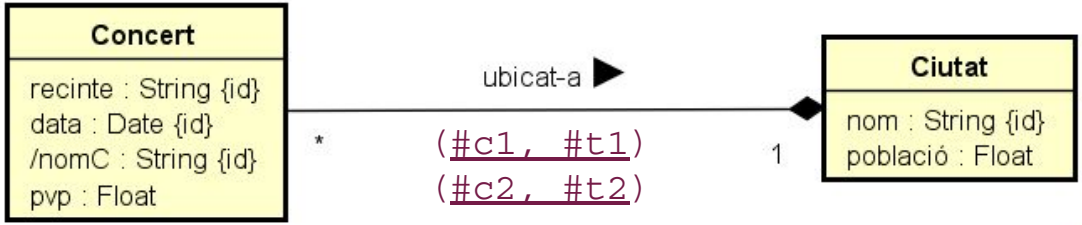
(#c1, #t1)
(#c2, #t2)

ara és vàlid

RIT1(c1, c2: Concert): si c1 i c2 tenen el mateix recinte i data, aleshores el nom de la ciutat amb què estan associats ha de ser diferent

Altres conceptes: clau externa (3)

Pot ser còmode definir un atribut derivat en la classe



(#c1, #t1)
(#c2, #t2)

(#t1, "BCN", 2M)
(#t2, "Madrid", 3M)

(#c1, "Platja", 1/8/23, "BCN", 15)
(#c2, "Platja", 1/8/23, "Madrid", 30)

ID1(c: Concert): c.nomC = nom de la ciutat associada a c

2.2.3 Model del comportament

Diagrames de seqüència i contractes

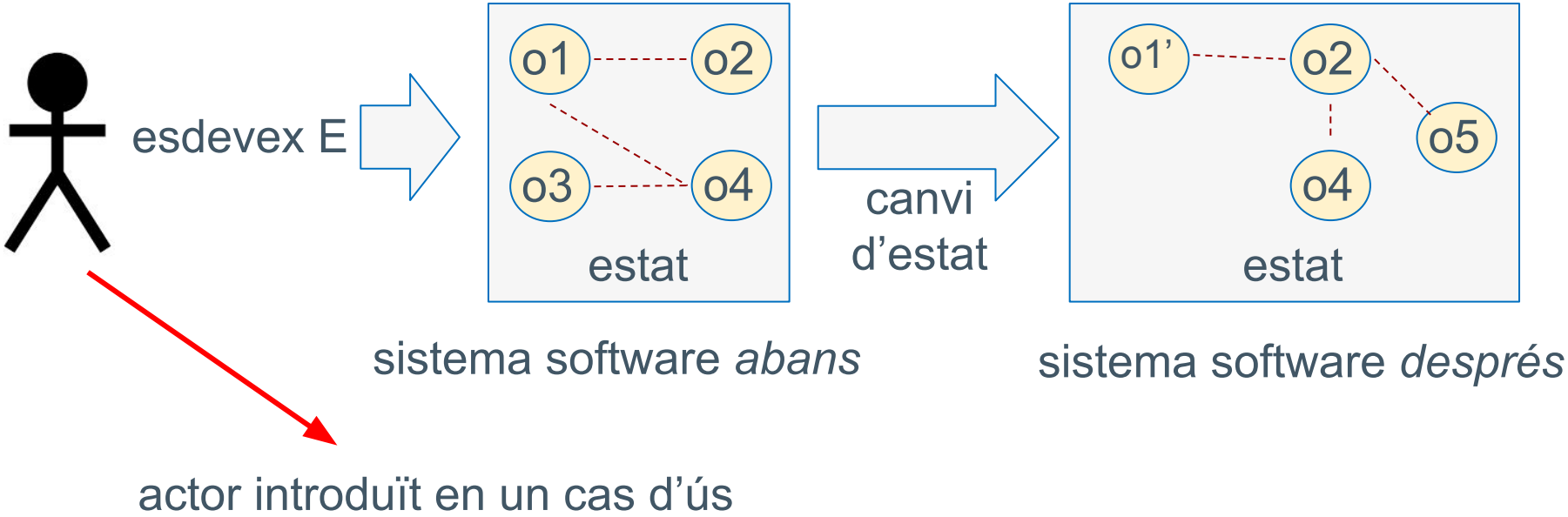
Model del comportament del sistema

El model del comportament ha de permetre descriure l'evolució del sistema al llarg del temps

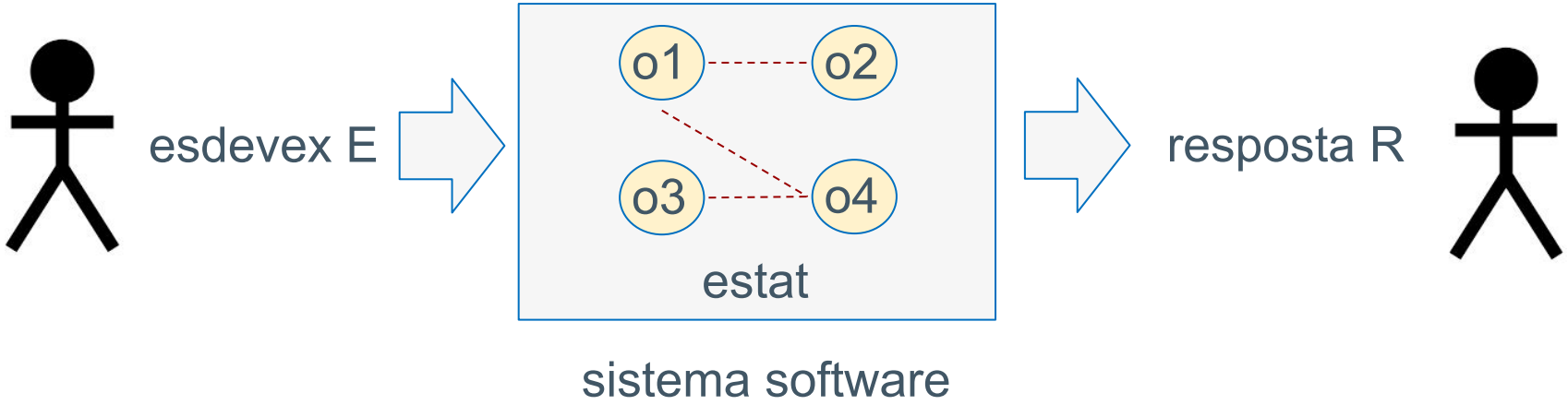
Ha de combinar adequadament els conceptes següents:

- **estat** (del sistema): col.lecció d'objectes concordants amb l'estructura i restriccions especificats al model de dades
- **esdeveniment extern (*esdevex*)**: interacció amb el sistema iniciada per un actor que ha de ser processat adequadament
- **resposta**: dades produïdes pel sistema en resposta a un esdevex
- **canvi d'estat**: modificacions en la col.lecció d'objectes (i els seus valors i associacions) que formen l'estat degut a un esdevex

Comportament del sistema: modificacions

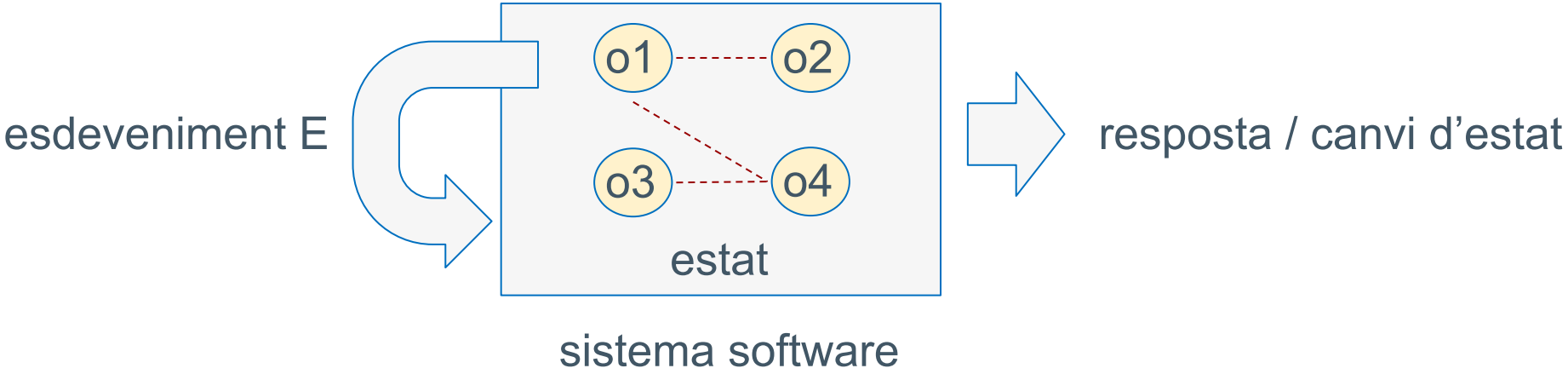


Comportament del sistema: consultes



Un mateix esdevex pot provocar tant modificacions com consultes

Comportament del sistema: esdeveniments auto-generats



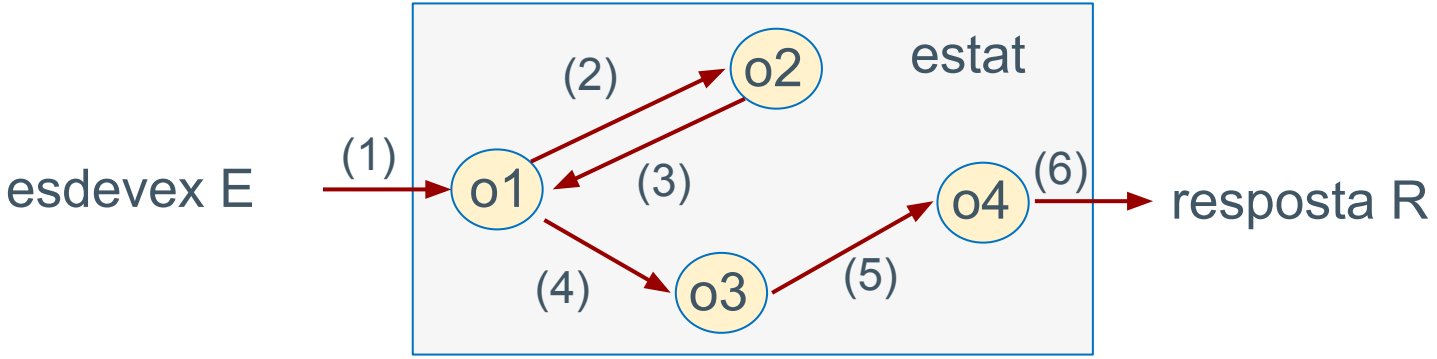
Per exemple, execució d'un esdeveniment diari

Per conveniència, els considerem també com a esdevex, i modelem el rellotge com a actor

Detall del comportament del sistema

Els objectes que formen l'estat interactuen en resposta als esdevexs per produir respostes i/o canviar d'estat

Si mirem què passa dins de l'estat (p.e., en una consulta):



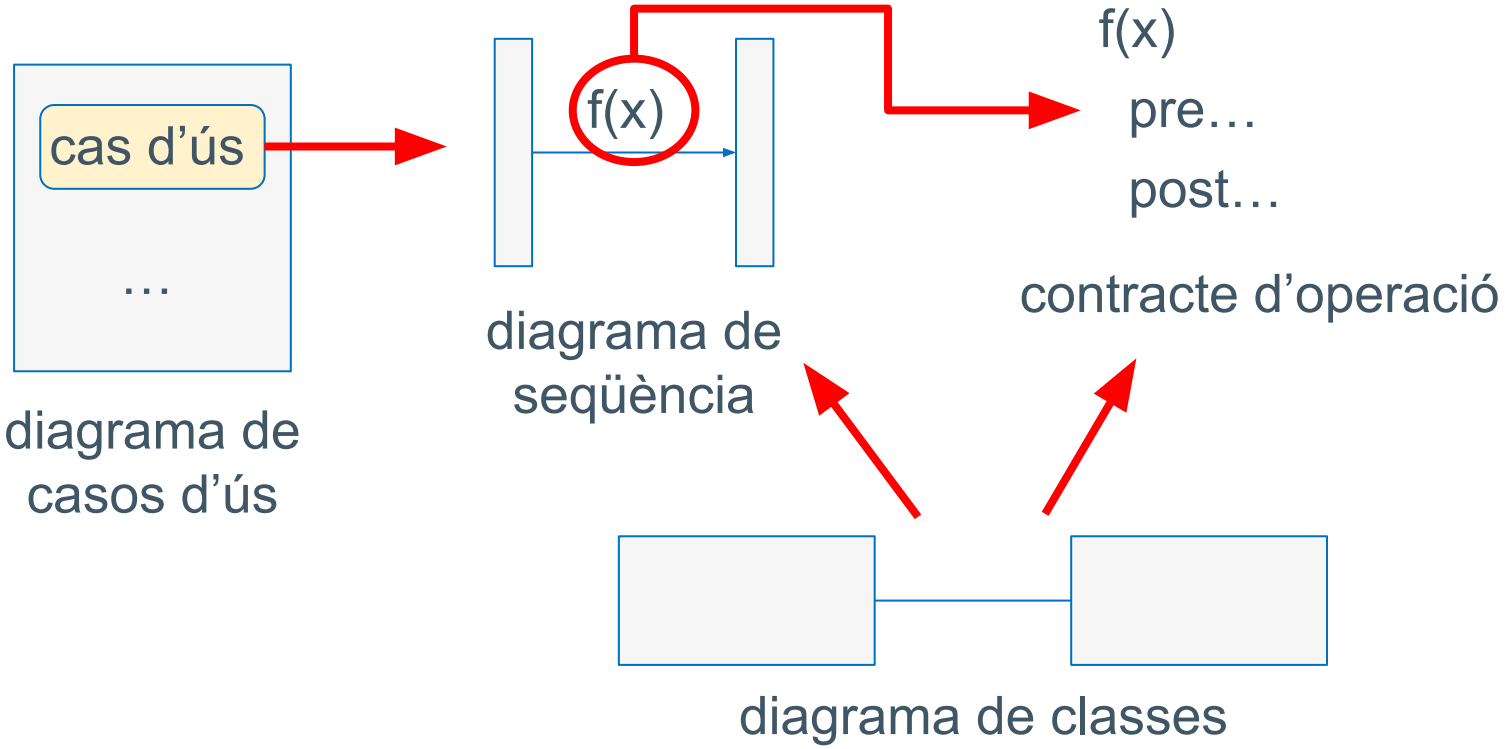
A l'especificació, però, **considerem el sistema con un únic objecte** que encapsula (amaga) tots els objectes components

Elements del model del comportament

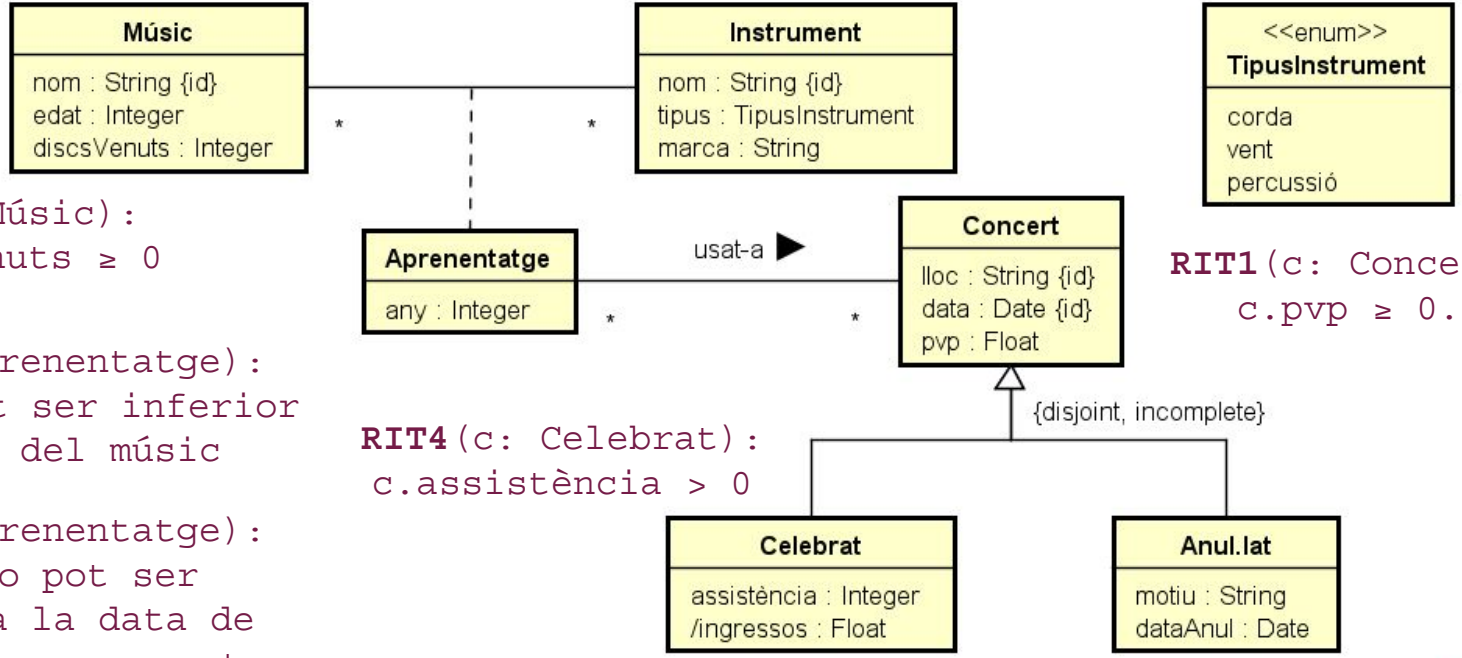
Per poder descriure aquesta comunicació entre objectes, UML ofereix dos conceptes de modelització:

- **operacions**: comunicacions vàlides entre objectes
  - es defineixen a les classes mitjançant **contractes**
  - les operacions estan vinculades a l'objecte únic que representa el sistema
    - aquesta visió canviarà a l'etapa de disseny
- **diagrames de seqüència**: determinen l'ordre d'execució de les operacions
  - són una mena de programa d'alt nivell
  - es defineixen a partir dels esdevex

Procés d'especificació del model del comportament



Exemple: diagrama de classes dels músics i concerts



RIT2(m: Músic):
m.discsVenuts ≥ 0

RIT1(c: Concert):
c.pvp ≥ 0.0

RIT5(d: Aprenentatge):
d.any no pot ser inferior a l'edat del músic

RIT4(c: Celebrat):
c.assistència > 0

RIT6(d: Aprenentatge):
d.any no pot ser superior a la data de cap dels seus concerts

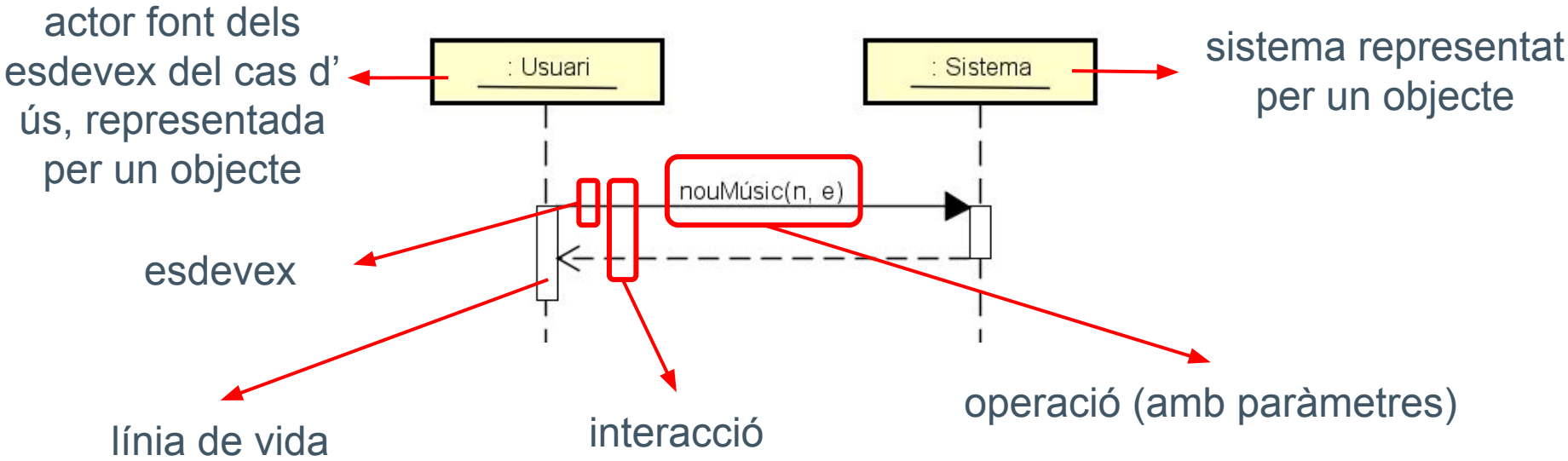
ID1(c: Celebrat):
c.ingressos = c.assistència * c.pvp

RIT3(a: Anul.lat):
a.dataAnul ≤ a.data

Diagrames de seqüència

Cada escenari rellevant del cas d'ús genera un diagrama de seqüència

- exemple: cas d'ús de donar d'alta un músic:



Contractes

Defineixen els efectes de les operacions identificades als diagrames de seqüència com a resposta als esdevex

```
operació nouMúsic (n: String; e: Integer)
pre e ≥ 0
pre no existeix al sistema cap Músic amb nom n
post crea una instància de Músic = (n, e, 0)
```

signatura

precondicions: condicions necessàries per a què l'operació es comporti correctament

postcondició: efecte de l'operació en cas que es compleixin les pre i suposant un estat de partida vàlid segons el model de dades

Redundància

Sobre-especificació de les condicions de correctesa

```
operació nouMúsic (n: String; e: Integer)
pre e ≥ 0
pre no existeix al sistema cap Músic amb nom n
post crea una instància de Músic = (n, e, 0)
```

ja apareixen al model de dades; estrictament, no és necessari consignar-les de nou

```
operació nouMúsic (n: String; e: Integer)
post crea una instància de Músic = (n, e, 0)
```

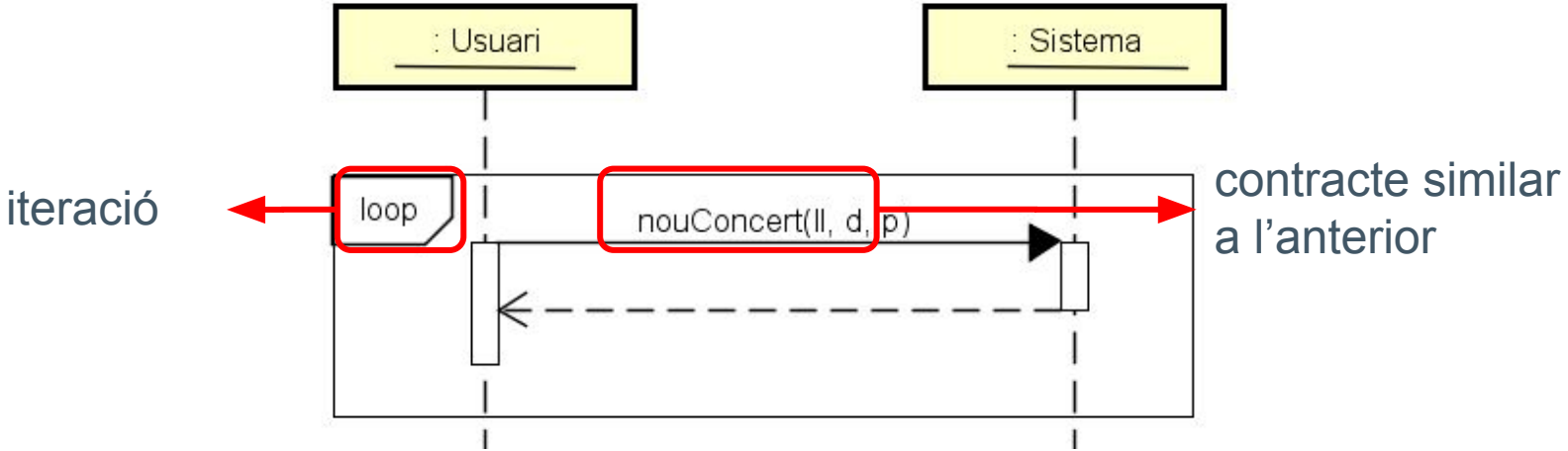
Afegint estructura en els diagrames de seqüència

Constructor bàsic: **marc** (***frame***)

- augmenten la capacitat expressiva dels diagrames de seqüència
- permeten expressar repeticions i alternatives

Marc iteratiu (loop)

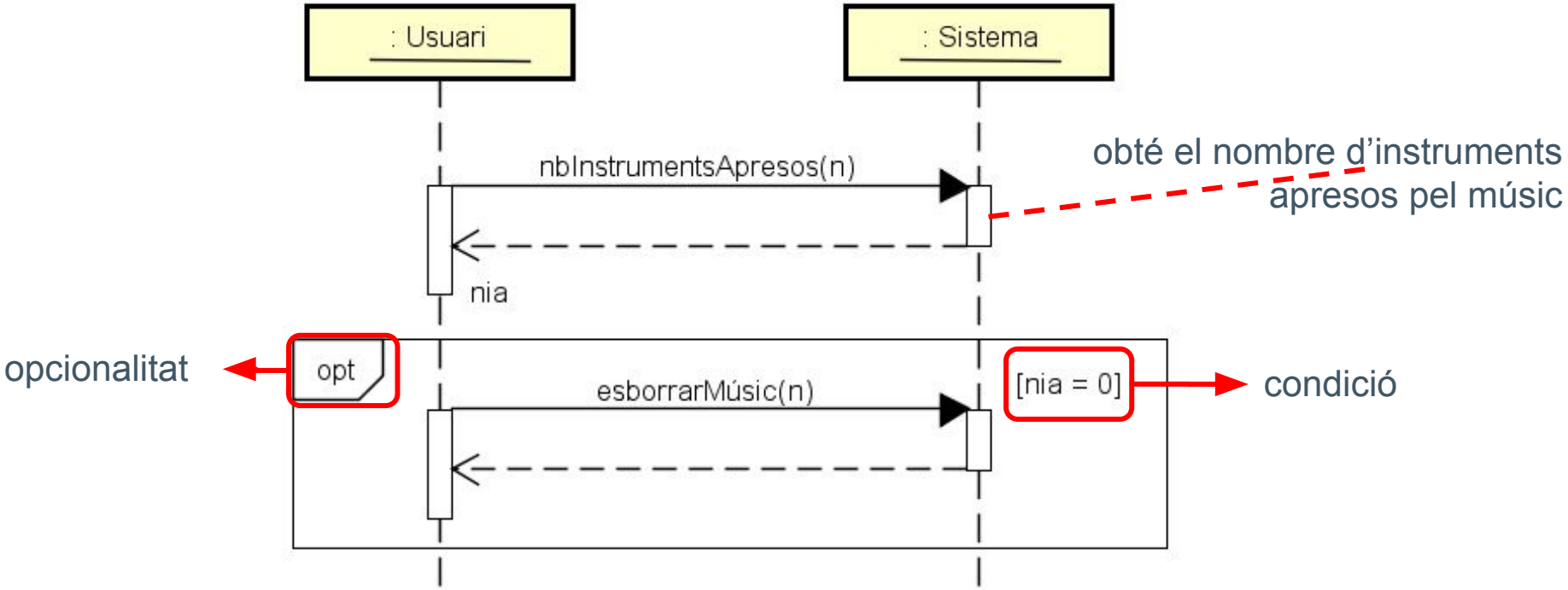
Exemple: cas d'ús de planificar gira d'un músic. L'Usuari entra en el Sistema tots els concerts de la temporada, amb el seu lloc, la seva data i preu de venda al públic de les entrades



Marc alternatiu (opt) - 1

Exemple: cas d’ús Purgar Músic: l’Usuari pregunta al Sistema si té constància que un músic sàpiga tocar almenys un instrument. Si no és el cas, l’esborra

Marc alternatiu (opt) - 2



Marc alternatiu (opt) - 3

```
operació nbInstrumentsApresos(n: String): Integer
pre existeix un músic m amb m.nom = n
retorna nombre de instàncies de Instrument associats amb m
```

més formalment: `tamany(m.instrument)`

funció predefinida

navegació per una associació

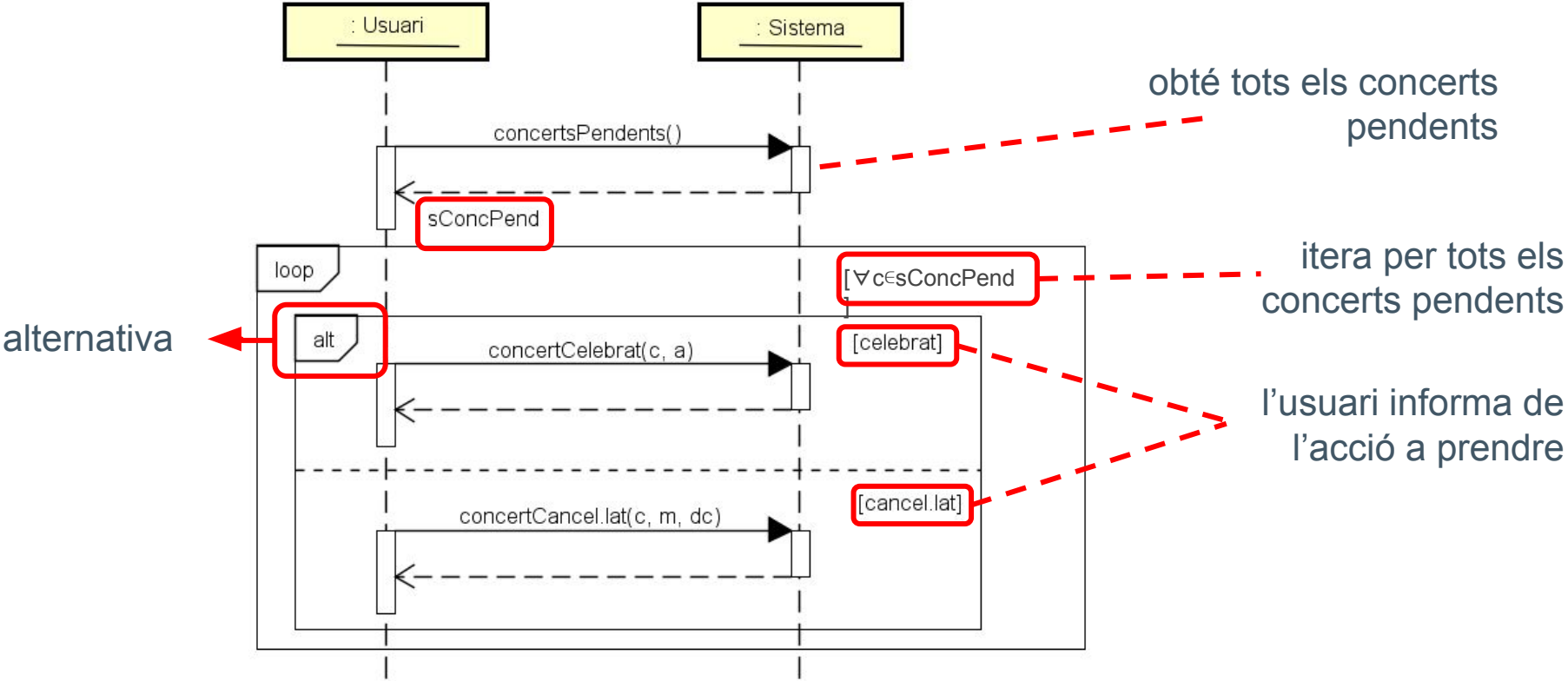
```
operació esborrarMúsic(n: String)
pre existeix un músic m amb m.nom = n
post no existeix un músic m amb m.nom = n
```

es pot considerar redundant perquè ja s'ha comprovat abans, però si ho posem és més fàcil de reusar

Marc alternatiu (alt) (1)

Exemple: cas d’ús Netejar Concerts: el Sistema presenta a l’Usuari tots els concerts amb data anterior a l’actual i que no estan ni cancel.lats ni celebrats i, per cadascun, l’Usuari entra el seu estat final, amb la informació corresponent en cada cas

Marc alternatiu (alt) (2)



Marc alternatiu (alt) (3)

```
operació concertsPendents(): Conjunt(Concert)
retorna tots els concerts c tal que tipus(c) = Concert i
        c.data > avui()
```

funció predefinida

```
operació concertCelebrat(c: Concert; a: Integer)
pre a > 0
pre tipus(c) = Concert
post tipus(c) = Celebrat amb c.assistència = a
```

especialització de l'objecte

redundant (assegurat per la lògica del diagrama); pot no posar-se

redundant (assegurat per RIT4); pot no posar-se

concertAnul.lat té un contracte similar

Compartint informació en els diagrames de seqüència

Sovint, els diagrames de seqüència requereixen l'execució de diverses operacions

- i pot ser necessari compartir informació entre operacions
- es poden usar resultats i paràmetres amb aquest objectiu

Hi ha dues estratègies bàsiques:

- anar repetint els mateixos paràmetres diverses vegades
- obtenir i compartir objectes com a forma més àgil de treballar

Ens centrem en aquesta segona estratègia

Compartició d'informació: diagrama de seqüència

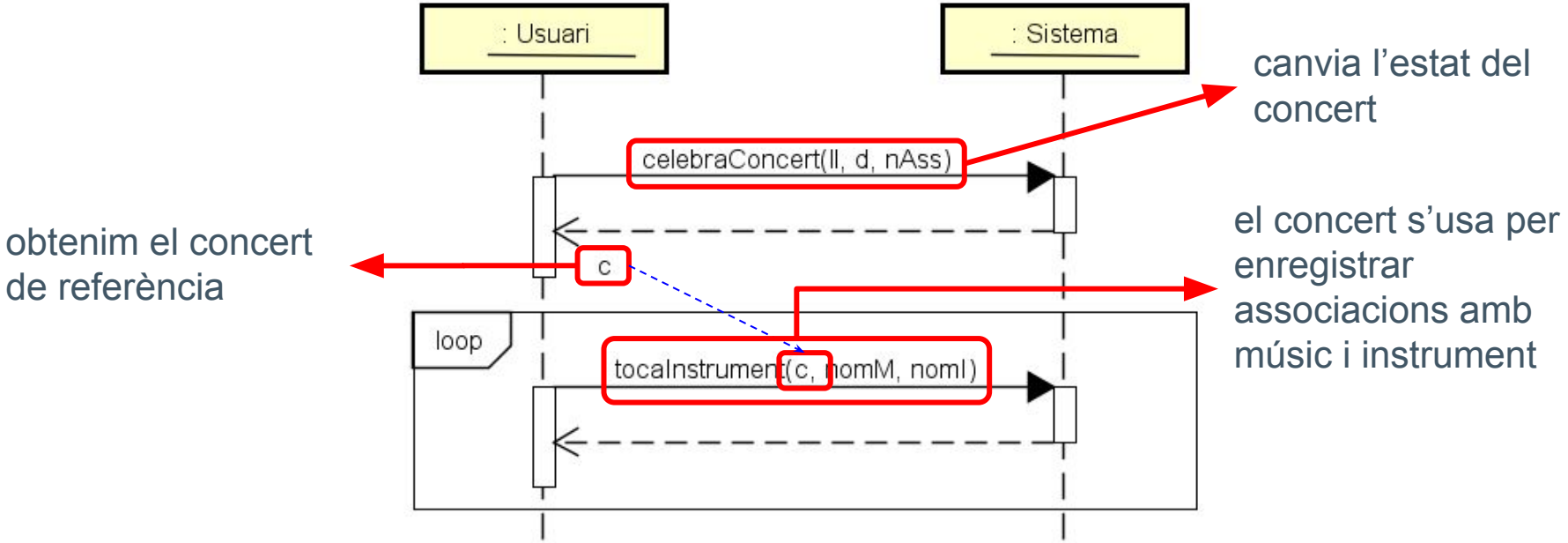
Exemple: cas d'ús per donar per celebrat un concert, enregistrant els músics que hi han tocat

Esdeveniments externs:

- primer, l'Usuari entra l'identificador del concert i el nombre final d'assistents
- després, un a un, entra els noms del músic i de l'instrument per enregistrar que l'ha tocat
- el Sistema ha d'anar controlant tots els possibles errors

Compartició d'informació: diagrama de seqüència

Exemple: cas d'ús per donar per celebrat un concert, enregistrant els músics que hi han tocat



Compartició d'informació: contractes (1)

```
operació celebraConcert(ll: String; d: Date; nAss: Integer): Concert
pre ∃c: Concert amb c.lloc = ll i c.data = d
pre tipus(c) = Concert
pre nAss > 0
post tipus(c) = Celebrat
post c.assistència = nAss
retorna c
```

es declara el resultat com a super-classe `Concert` i no de la subclasse `Celebrat`, per major reusabilitat de l'operació següent

notem que no cal la post de calcular el valor de l'atribut `ingressos` perquè és derivat, de manera que es calcula automàticament

redundant amb el model de dades; pot no posar-se

Compartició d'informació: contractes (2)

```
operació tocaInstrument(c: Concert; nomM, nomI: String)
pre tipus(c) ≠ Anul.lat
pre ∃m: Músic amb m.nom = nomM
pre ∃x: Instrument amb x.nom = nomI
pre ∃a: Aprenentatge que uneix m i x
pre a.any ≤ c.data.any
pre a i c no estan ja associats
post crea una instància u de l'associació usat-a, u = (a, c)
```

l'associació `usat-a` està definida sobre la superclasse `Concert`, i en aquest context l'objecte `c` sabem que és de subclasse `Celebrat`. Cap problema, doncs tota la informació de la superclasse és accessible a totes les seves subclasses

redundant; millor eliminar, per comoditat

tot i que en aquest context, segur, més val dir-ho per fer l'operació reutilizable

Bibliografia

- C. Larman. Applying UML and Patterns. An Introduction to Object-oriented Analysis and Design. 3a ed. Prentice Hall, 2005.
- J. Rumbaugh, I. Jacobson, G. Booch. The Unified Modeling Language Reference Manual. 2a ed. Addison-Wesley, 2004.
- Transparències de l'assignatura Introducció a l'Enginyeria del Software (IES), FIB-UPC (darrera edició: 2023).

Recursos

- https://www.uml.org